


# ФСП Талант
# Сопроводительная документация

Цифровая платформа-агрегатор ИТ-вакансий с верифицированным профилем достижений участника Федерации спортивного программирования. ЛЦТ 2026, раздел 3.4 технического задания.

Репозиторий: https://github.com/KonkovDV/fsp-talent
Сборка документа: 07.10.2026
Исходные тексты — Markdown в каталоге docs/, этот файл собран скриптом docs/tools/build_pdf.py.

# Содержание

1. Сопроводительная документация
2. Соответствие техническому заданию
3. Сборка, развёртывание и запуск
4. Проверка теста на грейд
5. Проверка подбора
6. Нагрузочная проверка
7. Интеграция с ФСП
8. Защита от фиктивных вакансий и ATS
9. Коды ошибок API
10. Дизайн-система

# ФСП Талант: сопроводительная документация

Документ отвечает на раздел 3.4 технического задания. Каждый пункт раздела — отдельная глава, порядок тот же. Все цифры получены запуском кода, команды приведены рядом.

| Пункт ТЗ 3.4 | Глава |
|---|---|
| Функциональная и компонентная архитектура | 1, 2 |
| Механика тестирования и устойчивость к распространению заданий | 3 |
| Механика подбора и логика категоризации | 4 |
| Процедура валидации и её результаты | 5 |
| Схема интеграции с реестром ФСП | 6 |
| Описание API | 7 |
| Библиотеки и компоненты с версиями | 8 |
| Инструкция по сборке, развёртыванию и запуску | 9 |

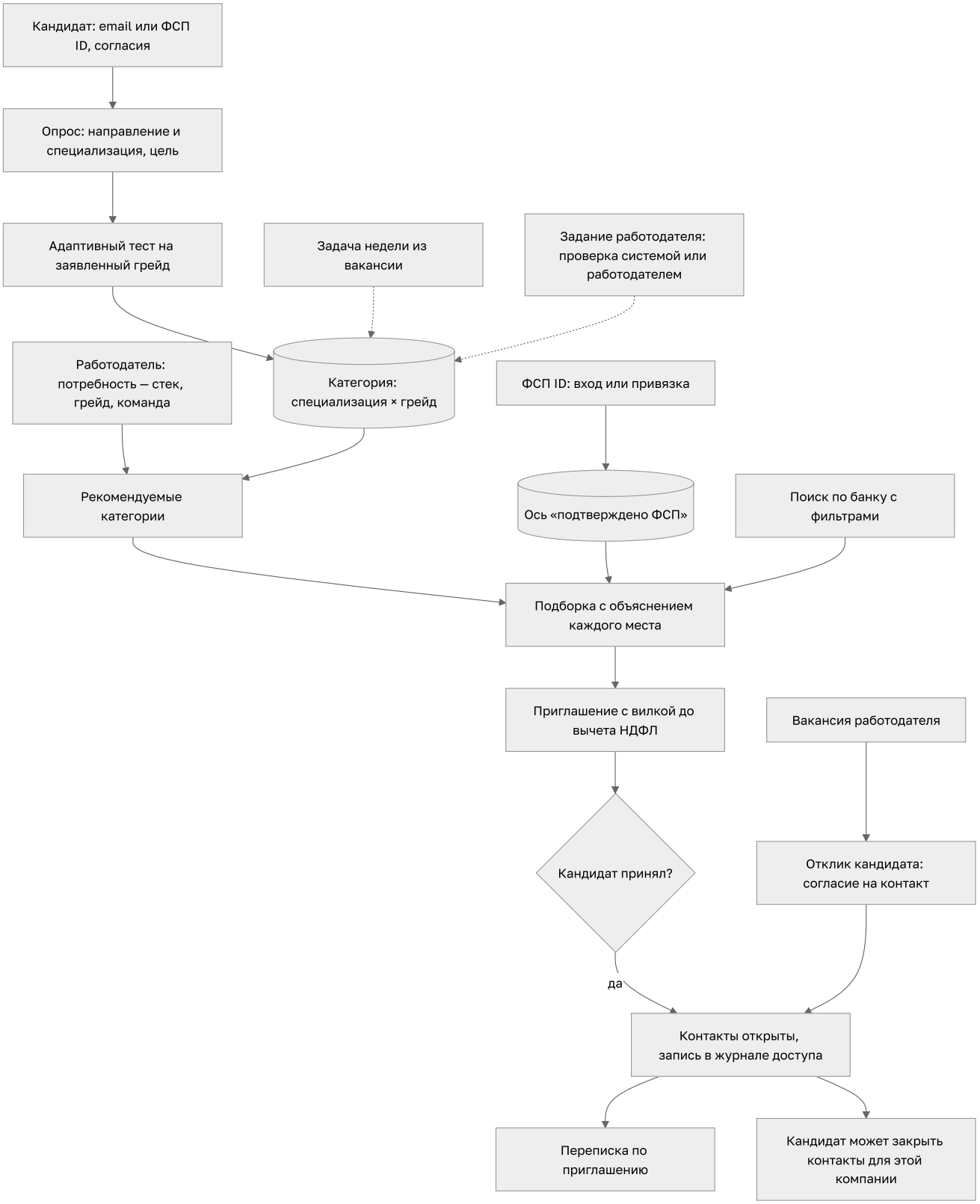
## 0. Решение в двух абзацах

Кандидат регистрируется по email или входит через ФСП ID, проходит опрос (направление и специализация, цель: работа или стажировка, предметная область по желанию), выбирает грейд и подтверждает его адаптивным тестом. Из этого складывается категория «специализация × грейд». Результаты соревнований ФСП не меняют грейд: это отдельная подтверждённая ось профиля. Кандидат без истории ФСП ничего не теряет.

Работодатель не ждёт откликов. Он описывает потребность: стек, специализацию, грейд, чем занимается команда. Платформа показывает, в каких категориях искать, и даёт подборку, где у каждого места есть объяснение. Работодатель сам отправляет приглашение с вилкой до вычета НДФЛ и может дать кандидату своё задание. Имя и контакты кандидата открываются только после того, как кандидат принял приглашение; дальше стороны переписываются на платформе, а кандидат может закрыть контакты для одной компании. Классический путь тоже есть: вакансии, отклик, статусы.

# 1. Функциональная архитектура

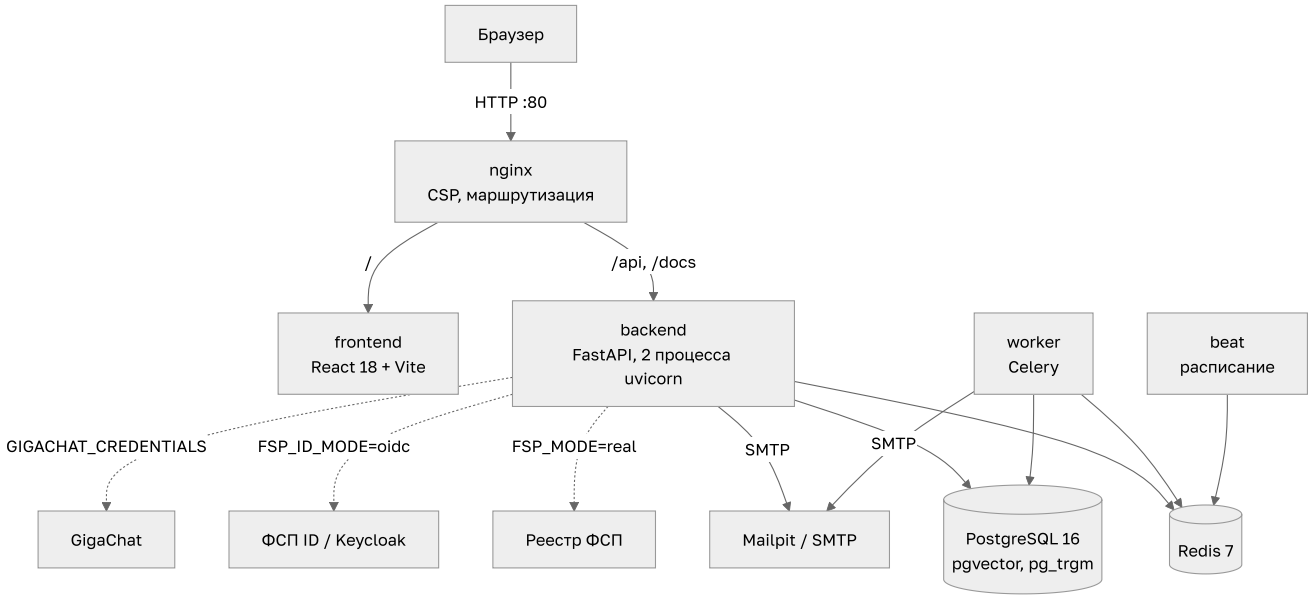
## Роли и сценарии



### Что умеет каждый кабинет

| Кандидат (ТЗ 2.1) | Работодатель (ТЗ 2.1) |
|---|---|
| Профиль вручную или из PDF-резюме: ФИО, контакты, стек, грейд, стаж, софт-скиллы. Каждое поле предлагается с цитатой и переносится по кнопке | Профиль компании: сфера, сайт, описание, ИНН с проверкой контрольной суммы. Превью приглашения глазами кандидата |
| Опрос, выбор грейда, адаптивный тест, итог по темам, история попыток | Потребность с версиями: изменения не теряют прежнюю подборку и отобранных |
| Категория и грейд, подтверждённые тестом. Пересдача на грейд ниже или выше с ограничением частоты. На стенде есть пробный проход из 15 заданий и 5 минут | Рекомендуемые категории и подборка, у каждого кандидата — разбор факторов и «лучше N % категории» |
| Задания от работодателей (общие для специализации и адресные), задача недели из вакансии и короткая практика по слабой теме | Поиск по всему банку: специализация, грейд, стек, дисциплина и разряд ФСП, только подтверждённые тестом |
| Входящие приглашения с вакансией, вилкой и профилем компании: принять или отклонить, переписка после принятия, закрыть контакты для одной компании | Сравнение до четырёх кандидатов рядом, без имён. Свои задания кандидатам: с эталоном (проверяет система) или открытые (проверяет работодатель) |
| Вакансии под профиль, отклик, статусы откликов | Приглашение без привязки к вакансии, статусы: отправлено, просмотрено, принято, отклонено, отозвано, истекло |
| Приватность: видимость карточки, согласия и их отзыв, удаление аккаунта, вход через ФСП ID и привязка | Вакансии с редактированием, отклики со сменой статуса |
| Уведомления на сайте и письма | Контакты только после согласия кандидата, переписка по принятому приглашению, кнопка «Какой тест получится» у вакансии и вкладка «Как проверим» у потребности |

## 2. Компонентная архитектура




Монолит с явными слоями (ТЗ 3.5): роутеры принимают запрос и проверяют права, бизнес-логика живёт в сервисах, которые не знают про HTTP, модели отвечают только за хранение. Внешние системы (реестр ФСП, ФСП ID, GigaChat, почта) подключены через адаптеры с флагом режима: в демо всё работает без внешних сервисов.

### Модули backend

| Слой | Каталог | Что внутри |
|---|---|---|
| API | `app/api/v1/` | 15 роутеров: auth, candidates, search, offers, messages, vacancies, needs, tasks, assessment, employer, legal, notifications, public, ai, fsp-id-demo. Единый формат ошибок `app/api/errors.py` |
| Тест | `app/items/`, `app/services/cat.py`, `assessment.py`, `item_monitor.py`, `practice.py`, `vacancy_exam.py` | Параметрические семейства заданий, IRT 3PL, адаптивный выбор, мониторинг утечки, задача недели, тест по вакансии, пробный проход |
| Подбор | `app/services/matching.py`, `search/pipeline.py`, `categorization.py`, `skills.py` | Оценка соответствия, SQL-фильтры, полнотекстовый ранг и RRF, категории, нормализация навыков |
| ФСП | `app/services/fsp/`, `app/services/scoring/` | Реестр (mock и HTTP), ФСП ID (OIDC), ось ФСП: класс результата, давность, байесовское сжатие |
| Доверие | `app/services/vc/`, `identity.py`, `privacy.py`, `notify.py`, `inn.py` | Удостоверения VC 2.0, коды подтверждения, согласия и удаление данных, уведомления, проверка ИНН |
| Задания и переписка | `app/services/tasks.py`, `app/api/v1/tasks.py`, `messages.py` | Задания от работодателя, проверка ответа, строка в карточке, переписка по приглашению |
| Фоновые задачи | `app/workers/` | Пересчёт профилей, истечение приглашений, задача недели, очистка по 152-ФЗ |
| Проверка | `app/validation/` | Симуляция теста, проверка подбора, калибровка заданий |
| Данные | `app/models/`, `alembic/versions/` | одна начальная миграция, у неё есть откат |

### Данные

Главные таблицы: `participants`, `skills`, `results` (строки протоколов ФСП), `discipline_scores` (ось ФСП), `category_assignments`, `test_sessions` и `test_session_items`, `grade_confirmations`, `item_stats` (статистика ключей заданий), `practice_attempts`, `employer_tasks` и `task_submissions` (задания работодателя и ответы), `employers`, `needs` и `need_versions`, `need_shortlist`, `offers`, `offer_messages`, `vacancies`, `applications`, `consents`, `consent_logs`, `access_logs`, `notifications`, `verifiable_credentials`.

## 3. Механика тестирования

ТЗ отвергает два решения: единый набор заданий (ответы разойдутся по рынку) и генерацию нейросетью под каждого (сложность несравнима). Мы используем третий путь.

**Параметрические семейства.** Семейство — шаблон задания с параметрами и кодом, который вычисляет правильный ответ. Каждый показ — новый изоморф: другие числа, строки и фрагменты кода. В оценке изоморфы одного уровня делят родителя. Различимость `a` — априор семейства, по ответам в банк она не записывается. Трудность `b` задаёт уровень (от −2,2 до 1,8). Страница «Как устроен тест» показывает `a` каждого семейства. Сложность не берётся из случайных чисел конкретного показа. Семнадцать семейств с вычисляемым ответом (Python, SQL, JavaScript, алгоритмы, графы, сети, безопасность, ML-метрики, встраиваемые системы, границы и расхождения тестов, метрики и доли аналитика, непокрытые требования и шаги цепочки, пакеты и опоздания в конвейере данных) и пулы вопросов на понимание: у специализаций с вопросами на понимание не меньше шести идей, у каждой идеи несколько формулировок. У тестирования план из четырёх семейств с вычисляемым ответом. У аналитика их шесть, у инженерии данных четыре плюс вопросы на понимание конвейера. Юнит-тест `test_item_families.py` перезапускает каждый сгенерированный фрагмент в Python, Node.js, Go, Java, Kotlin, C# или SQLite и сверяет ключ. Языковая модель во время теста не используется.

**Почему утечка не помогает:**

- выученный ответ не подходит к новым параметрам;
- если ответ — целое число, его вводят с клавиатуры, вариантов на экране нет: угадать нельзя;

- правильный ответ не уходит в браузер до конца теста, ответ принимается только на текущее задание по HMAC-токену, время засекает сервер. Ключ в строке задания подписан отдельно: если номер правильного варианта в базе не сходится с подписью, ответ не входит в θ. Повтор той же отправки несёт Idempotency-Key и не открывает вторую попытку;
- если доля верных ответов по ключу растёт сверх ожидаемой по способностям кандидатов ($z \geq 3{,}5$) или то же показывает накопленная сумма избытка, ключ уходит в карантин. Статистика открыта: `GET /api/v1/public/assessment/items`;
- вторая попытка не повторяет вопросы на понимание: общих не больше 40 %.

**Адаптивность.** Модель IRT 3PL, оценка способности EAP. Следующее задание выбирается по информации в точке порога заявленного грейда. Среди пяти лучших срабатывает Симпсон–Хеттер: пока по ключу нет измеренной экспозиции, выбор равномерный, а ключ, который уже брали чаще целевой доли 0,40, берётся с вероятностью «цель / доля». Учитывается план тем специализации. Тест идёт от 8 до 20 заданий и заканчивается, когда вероятность уровня выходит за 10 % или 90 %. Если к этому моменту вопрос о следующем уровне уже закрыт, раньше может сработать обобщённое отношение правдоподобия у порога (зона ±0,25, порог ln 9). Во время теста кандидат видит эту уверенность и оценку θ ± погрешность. После основного теста, если ответы плохо сходятся с оценкой (поправка Снейдерса ниже −1,645), платформа предлагает пересдать. Это не вывод о списывании. Быстрый верный ответ на задание с кодом (меньше четырёх секунд) копится по семейству: три таких ответа в одном семействе помечают попытку для разбора. Грейд и карантин ключа от этого не меняются. Кривые информации у порогов, погрешность и показы по семействам открыты на странице «Как устроен тест» (`/method`).

**Решение о грейде.** Грейд даёт только основной тест специализации: не тест по вакансии и не результаты ФСП. Кандидат сам выбирает уровень, тест его подтверждает или нет; грейд подтверждён при вероятности уровня от 75 %. Тест на уровень подтверждает только этот уровень. Если результат уверенно выше (от 90 % за следующий порог), платформа предлагает отдельный тест на ступень выше, но сама грейд не повышает. Подтверждённый грейд не понижается (ТЗ 2). Не сдали — тест на уровень ниже доступен сразу, попытка не сгорает. Пересдача того же уровня — через 7 дней, прерванная попытка засчитывается. После смены грейда следующее повышение — через 90 дней. Для стажировки есть грейд «стажёр» на ступень ниже junior.

**Пробный проход на стенде.** Когда включён демо-режим, кандидат может пройти 15 заданий за 5 минут (по 20 секунд на задание). Итог подписан как пробный: грейд не подтверждается, категория не создаётся, поиск эту попытку не видит.

**Тест зависит от вакансии.** Навыки, названные в тексте вакансии или потребности, подтягивают свои семейства (`vacancy_exam.py`). Работодатель до публикации видит доли тем, цитаты, по которым определены навыки, и три примера заданий без ответа. Кандидат может пройти такой тест: он покажет результат по темам вакансии и подтвердит навыки, но грейд не изменит. В демо девять вакансий с описаниями: на них видно, как разные тексты дают разный тест.

**Регулярные задания от работодателя (ТЗ 2.1).** Федерация уточнила, что регулярное задание пишет работодатель. Работодатель создаёт задание в разделе «Задания» или кнопкой «Дать задание» в карточке кандидата. Тип ответа — число (эталон — целое), выбор одного из 2–8 вариантов или открытый текст до 4000 знаков (решение или подход к решению). Число и выбор проверяет система сразу, эталон кандидату не показывается, пока он сам не ответил. Открытый ответ получает статус «на проверке», работодатель ставит «зачтено / не зачтено» с комментарием. Задание можно выложить для всех кандидатов специализации или дать одному кандидату из поиска или подборки (по анонимному id карточки), можно привязать к вакансии и задать срок. Один ответ на задание, не больше 30 заданий в сутки от компании. Вердикт появляется в анонимной карточке строкой «Задание работодателя «…»: зачтено, вчера». Грейд задание не меняет. Уведомления (колокольчик и письмо) не содержат данных другой стороны.

**Задача недели.** Кроме заданий работодателя, раз в неделю планировщик выбирает каждому активному кандидату открытую вакансию его специализации и присылает «задачу недели», собранную платформой из текста вакансии на тех же семействах. Грейд она не меняет, работодатель видит в карточке тему, результат и давность. После теста платформа ещё предлагает практику по самой слабой теме.

Подробно, с формулами и таблицами: VALIDATION_ASSESSMENT.md.

## 4. Механика подбора и категоризация

**Категория** — специализация из опроса и грейд. Грейд берётся из теста по этой специализации. До теста грейд заявленный: в выдаче он помечен как заявленный и в оценке соответствия весит 0,85 от подтверждённого, поэтому кандидат с неподтверждённым грейдом стоит ниже такого же с подтверждённым. Справочник: одиннадцать специализаций (backend, frontend, fullstack, mobile, ML и системы ИИ, DevOps, информационная безопасность, встраиваемые системы и робототехника, тестирование, аналитика, инженерия данных) и пять грейдов (стажёр, junior, middle, senior, lead).

**Ось ФСП** считается отдельно (версия методики `v3.1.0`) и влияет на место внутри категории, но не на грейд. Все соревнования весят одинаково, статус не учитывается. Место в протоколе → класс результата: победитель 1,00, призёр 0,85, топ-10 % 0,65, верхняя половина 0,45, участие 0. Давность → затухание с полупериодом 18 месяцев, командный результат → вес доказательства `1/√m`. В сумму дисциплины входят пять сильнейших событий по произведению качества и веса, остальные остаются в профиле и скор не двигают. По дисциплине — байесовское сжатие к среднему и ранжирование по нижней границе LCB = μ − σ: один случайный успех не обгоняет стабильную серию, а участия без призов не поднимают среднее. Разряд-эквивалент по ЕВСК показывается для справки и в оценку не входит.

**Порядок выдачи** задаёт одна оценка соответствия, разложимая на части (`app/services/matching.py`):

- попадание в список решает соответствие потребности: навыки 0,50, грейд 0,20, специализация 0,20, текст 0,05, предметная область 0,05;
- навык весит по силе доказательства: подтверждён тестом 1,0, найден цитатой в резюме 0,75, заявлен 0,55;
- среди кандидатов с одинаковым соответствием выше тот, у кого сильнее тест и протокол ФСП (ТЗ 2.1: «выше находятся те, у кого сильнее подтверждённый профиль и есть опыт участия в соревнованиях ФСП»);
- неприменимый фактор не учитывается: нет истории ФСП — ось выпадает, кандидат не теряет баллы;
- если и потребность, и кандидат назвали вилку до вычета НДФЛ, итоговое место — гармоническое среднее соответствия и вероятности, что кандидат примет вилку. Пока одна из сторон вилку не назвала, этот множитель не входит и таблица подбора не меняется. Вилка, которая не дотягивает до ожидания, опускает карточку и пишет, на сколько процентов. Если поднять верх вилки на 15 % закроет ожидание ещё хотя бы одному человеку и таких ожиданий не меньше пяти, подборка говорит, до какой суммы и скольких это коснётся. Прежний отказ из-за вилки снижает эту вероятность (множитель 0,85, не больше трёх раз);
- когда у подтверждения есть погрешность, фактор теста берёт вероятность на нижней границе $\hat{\theta} - 1{,}64 \cdot SE$. Если у сессии есть число верных ответов, туда же входит нижняя граница Уилсона этой доли, и фактор оставляет меньшую из двух. На стенде это число читает агрегат `app_answer_counts`: работодатель видит счётчик, а не текст задания и не ключ. В синтетике погрешности нет, а доля берётся как 12 ответов с числом верных, округлённым из вероятности уровня. Доля верных по теме на экране результата считается тем же интервалом и место в списке не задаёт;
- кандидат без теста занимает одно место в каждом десятке, если его оценка не хуже последнего в десятке больше чем на 0,08. Уже стоящий в десятке остаётся на своём месте;
- больше двух открытых приглашений опускает карточку на 0,02 за каждое следующее, но не больше чем на 0,10;
- имя, пол, возраст и регион в оценку не входят. Скрытая группа с теми же профессиональными фактами проходит правило четырёх пятых. Это проверяет тест `test_fairness.py`. В выборе заданий теста региона тоже нет: на `python -m app.validation.assessment --n 2000 --seed 7` четыре метки назначены отдельным генератором, максимальная доля одного ключа в группах 33 %, 35 %, 33 % и 37 %, отношение меньшей к большей 0,88.

**Объяснимость.** В карточке — строки «плюс / минус / инфо» по каждому фактору, у каждой строки машинный код и, где источник однозначен, метка «тест», «ФСП», «заявлено», «вилка» или «очередь». Рядом веса, «лучше N % кандидатов категории», результат задачи недели и вердикт по заданию работодателя с давностью.

Сравнение до четырёх кандидатов ставит факторы рядом. Если кого-то из выбранных уже нет в выдаче, сравнение отвечает 404. Если один обязательный навык — единственный пробел хотя бы у одного человека и в пуле не меньше пяти, подборка называет этот навык.

**Уточнение без потери подборки (ТЗ 2.1).** Потребность хранится версиями. Отобранные кандидаты помнят, по какой версии их выбрали, и остаются в списке после правки фильтров. Прошлую версию можно вернуть: её параметры становятся новой версией, список отобранных не трогается. Правка с устаревшим номером версии отклоняется.

**Приглашение.** Вилка ниже ожиданий кандидата показывает подсказку и не блокирует отправку. Повтор той же отправки с тем же ключом не создаёт второе приглашение. Одно ожидающее приглашение на пару работодатель–кандидат. Отказ без причины не сохраняется: зарплата, формат, стек, «не ищет» или другое. Доли причин и квартили ожиданий по категории показываются, когда в ячейке не меньше пяти человек. Условия отправленного приглашения не редактируются.

**Поиск по банку.** Сначала SQL отсекает закрытые профили, несовершеннолетних без согласия представителя, неподходящие специализацию, грейд, дисциплину и разряд. Затем полнотекстовый ранг PostgreSQL и триграммы объединяются методом RRF (K = 60) с рангом по результату теста. Okapi BM25 посчитан отдельно (`app/services/search/bm25.py`) и в эту выдачу не входит, поэтому таблица подбора от него не меняется. Смысловой поиск по эмбеддингам включается флагом `EMBEDDINGS_MODE=model` и на стенде выключен: модель sentence-transformers, файл ONNX в образ не кладётся.

## 5. Процедура валидации и результаты

Размеченных данных у хакатона нет (ТЗ 2), поэтому валидация построена на синтетике **со скрытой правдой**: модель видит только зашумлённые наблюдения, а качество считается по правде, которую модель не видела. Все проверки детерминированы и запускаются одной командой.

| Что проверяем (критерий ТЗ 6.2) | Результат | Команда (из `backend/`) |
|---|---|---|
| Соответствие эталону: точность решения вдали от границы грейда, 2000 кандидатов | 91,5 % | `python -m app.validation.assessment --n 2000 --seed 7` |
| Согласованность: каппа Коэна повторной сдачи с новыми заданиями | 0,51 | та же |
| Дискриминативность: медиана точечно-бисериальной корреляции, вводимый ответ | 0,22 | та же |
| Устойчивость к распространению: утечка у 30 % кандидатов, точность вдали от границы | 91,7 % с карантином ключей | та же |
| Угадывание проходит тест | 0 % | та же |
| Точность классификации по Руднеру у порога заявленного грейда | 83,5 % | та же |
| Перекрытие заданий между кандидатами одной симуляции | 15,2 % | та же |
| Подбор против поиска по ключевым словам, nDCG@10 (два набора) | +0,046 и +0,087 | `python -m app.validation.matching`, `--seed 7` |
| Справедливость: смена имени, пола, региона не меняет порядок | выполняется | `pytest tests/unit/test_fairness.py` |
| Нагрузка: 40 пользователей одновременно | 0 ошибок, p95 поиска 69 мс | `python tests/load_api.py` |

CI при каждом изменении повторяет проверку теста на 300 кандидатах и подбора (`--check`) и падает, если нарушен хотя бы один порог. На стенде ту же проверку запускает `make evaluate`. Жюри может проверить подбор на своей разметке: `python -m app.validation.matching --pairs ../docs/examples/jury_pairs.jsonl`.

Слепая разметка видит только заявленный профиль и в одном случае из пяти сдвигает оценку на ступень. Против меток, посчитанных по скрытой правде, каппа Коэна 0,46 при зерне разметчика 2027 (`python -m app.validation.matching --check`). Это другое зерно, чем зерно набора, и веса подбора по этой каппе не подгонялись. Набор профилей жюри лежит в `app/validation/resources` и проверяется командой `python -m app.validation.jury_pack app/validation/resources/jury_profiles.json` (и тем же CSV). В базу он не пишется. Страница «Режим жюри» (`/jury`) ведёт к двум демо-входам и к публичным страницам: отдельной роли жюри нет. `make evaluate` гоняет эту проверку вместе с тестом на 300 кандидатах (зерно 7), подбором, набором жюри и полными прогонами теста на 2 000 и 5 000. В CI остаётся короткий прогон на 300: полные таблицы длиннее обычной проверки стенда.

Отдельный прогон `python -m app.validation.assessment --n 5000 --seed 7` таблицу выше не заменяет и в пороги CI не входит. На нём точность вдали от границы 92,4 %, каппа пересдачи 0,48, точность по Руднеру 83,6 %, перекрытие заданий 16,9 %.

Подробно: VALIDATION_ASSESSMENT.md, VALIDATION_MATCHING.md, LOAD_TEST.md.

## 6. Интеграция с реестром ФСП

- **Вход через ФСП ID** — OpenID Connect, поток кода с PKCE. Платформа — конфиденциальный клиент Keycloak: пароль ФСП вводится только на странице провайдера, ФСП ID приходит в проверенном ID-токене. В демо роль Keycloak играет помеченная страница этого сайта с тем же протоколом.
- **Реестр** — один метод только на чтение: строки протокола участника. Пол, контакты и документы не запрашиваем. В демо — синтетический реестр, в режиме `FSP_MODE=real` — HTTP-клиент по предложенному контракту.
- **Удостоверения** — каждая новая строка протокола выдаётся как W3C Verifiable Credential 2.0 типа OpenBadgeCredential с подписью `eddsa-jcs-2022`. У хранимого удостоверения есть статус отзыва (`GET /api/v1/public/credentials/{id}/status`): подпись при отзыве не переписывается, статус начинает отвечать, что удостоверение отозвано. Краткая карточка подписывает только год, статус соревнования и корзину места, без названия события и без имени. Проверка: http://localhost/badge.

Схема обмена, контракт, настройка Keycloak и дельта-синхронизация: FSP_INTEGRATION.md.

## 7. API

Спецификация OpenAPI 3.1: http://localhost/openapi.json, Swagger UI: http://localhost/docs. Swagger отдаётся с того же адреса, без внешних CDN и встроенных скриптов, поэтому работает при строгой политике `script-src 'self'`. У каждого метода описаны запрос, ответ и коды ошибок.

| Группа | Методов | Что делает |
|---|---|---|
| candidates | 14 | Профиль, опрос, видимость, PDF-резюме и PDF-профиль, приглашения, закрытие контактов для компании, привязка ФСП ID |
| auth | 13 | Регистрация по email с кодом, вход, вход через ФСП ID, сброс пароля, обновление токена |
| assessment | 12 | Тест на грейд, история, разбор, практика и задача недели |
| needs | 11 | Потребности с версиями, подборка, сравнение, тест по потребности, отобранные |
| vacancies | 11 | Вакансии, редактирование, отклики и их статусы, контакты по отклику |
| tasks | 7 | Задания от работодателя: создание, ответы, проверка открытого ответа, закрытие; задания для кандидата |
| public | 8 | Удостоверения и их проверка, публичный профиль, рейтинг, статистика заданий |
| legal | 5 | Тексты согласий, выдача и отзыв согласий |
| offers | 4 | Приглашения, отзыв, контакты после согласия |
| notifications | 3 | Колокольчик: список и прочтение |
| employer | 2 | Профиль компании |

| Группа | Методов | Что делает |
|---|---|---|
| messages | 2 | Переписка по принятому приглашению |
| ai | 2 | Разбор резюме и текста запроса |
| search | 1 | Поиск по банку кандидатов |
| fsp-id-demo | 1 | Демо-провайдер ФСП ID |
| health | 1 | Проверка здоровья |

Всего 97 методов. Счёт по первому тегу метода в спецификации OpenAPI.

Ошибки приходят в одном формате с машинным кодом: `{"error": {"code", "message", "details"}}`. Полный список кодов: API_ERRORS.md, его сверяет с кодом юнит-тест.

## 8. Библиотеки и компоненты

Версии, на которых проходят тесты. Backend ставит их из `backend/requirements.lock`, frontend — из `frontend/package-lock.json`.

### Backend (Python 3.12)

| Библиотека | Версия | Лицензия | Зачем |
|---|---|---|---|
| FastAPI | 0.142.2 | MIT | HTTP API, OpenAPI |
| Starlette | 1.7.0 | BSD-3 | ASGI-основа FastAPI |
| Uvicorn | 0.54.0 | BSD-3 | ASGI-сервер |
| Pydantic / pydantic-settings | 2.13.5 / 2.15.0 | MIT | Схемы, валидация, настройки |
| SQLAlchemy | 2.1.3 | MIT | ORM, асинхронный доступ |
| asyncpg | 0.31.0 | Apache-2.0 | Драйвер PostgreSQL |
| Alembic | 1.20.0 | MIT | Миграции |
| pgvector (Python) | 0.5.0 | MIT | Тип vector для эмбеддингов |
| redis-py | 6.4.0 | MIT | Лимиты, состояние входа ФСП ID |
| Celery | 5.6.3 | BSD-3 | Фоновые задачи и расписание |
| python-jose | 3.5.0 | MIT | JWT и проверка ID-токена ФСП ID |
| bcrypt | 5.0.0 | Apache-2.0 | Хеши паролей |
| cryptography | 50.0.2 | Apache-2.0 / BSD | Ed25519 для удостоверений |
| httpx | 0.28.1 | BSD-3 | Клиент реестра ФСП и Keycloak |
| pypdf | 6.19.0 | BSD-3 | Текст из PDF-резюме |
| fpdf2 | 2.8.9 | LGPL-3.0 | PDF-профиль кандидата |
| pytest | 9.1.1 | MIT | Тесты |
| gigachat (необязательно) | ≥ 0.1.39 | MIT | ИИ-советник для резюме |
| sentence-transformers, torch (необязательно) | ≥ 3.0, ≥ 2.3 | Apache-2.0, BSD-3 | Смысловой поиск |

### Frontend (Node.js 20)

| Библиотека | Версия | Лицензия | Зачем |
|---|---|---|---|
| React, React DOM | 18.3.1 | MIT | Интерфейс |
| React Router | 6.30.6 | MIT | Маршруты |
| TanStack Query | 5.104.1 | MIT | Запросы и кеш |
| Zustand | 4.5.7 | MIT | Состояние входа |

| Библиотека | Версия | Лицензия | Зачем |
|---|---|---|---|
| React Hook Form, Zod, @hookform/resolvers | 7.89.0, 3.25.76, 3.10.0 | MIT | Формы и проверка |
| Axios | 1.20.0 | MIT | HTTP-клиент |
| lucide-react | 0.447.0 | ISC | Иконки |
| date-fns, clsx, tailwind-merge | 3.6.0, 2.1.1, 2.6.1 | MIT | Утилиты |
| @fontsource-variable/jetbrains-mono, golos-text | 5.3.0 | OFL-1.1 | Шрифты брендбука, без внешних запросов |
| Vite, @vitejs/plugin-react | 6.4.3, 4.7.0 | MIT | Сборка |
| TypeScript | 5.9.3 | Apache-2.0 | Типы |
| Tailwind CSS, PostCSS, Autoprefixer | 3.4.19, 8.5.28, 10.6.1 | MIT | Стили |

### Инфраструктура

| Компонент | Версия | Лицензия |
|---|---|---|
| PostgreSQL + pgvector + pg_trgm | 16 (образ `pgvector/pgvector:pg16`) | PostgreSQL License |
| Redis | 7.2 (`redis:7.2-alpine`) | BSD-3 |
| nginx | `nginx:alpine` | BSD-2 |
| Mailpit | `axllent/mailpit` | MIT |
| Caddy (публичный стенд) | 2 (`caddy:2-alpine`) | Apache-2.0 |
| Swagger UI | 5.33.1 (в репозитории) | Apache-2.0 |

Всё — открытое ПО (ТЗ 3.5). Исходный код решения открыт, не скомпилирован и не обфусцирован, лицензия MIT (LICENSE).

## 9. Сборка, развёртывание и запуск

```
git clone https://github.com/KonkovDV/fsp-talent.git && cd fsp-talent
make demo          # Windows: см. docs/INSTALL.md, три команды PowerShell
```

Сайт http://localhost, API http://localhost/docs, письма http://localhost:8025. Пошаговая инструкция с требованиями, проверкой, тестами, сбросом и частыми проблемами: INSTALL.md. Публичный стенд с HTTPS: deploy/README.md.

## 10. Безопасность и 152-ФЗ

- Отдельные согласия на обработку (ч. 1 ст. 9 152-ФЗ) и на распространение для публичной страницы (ст. 10.1). В базе хранятся версия и SHA-256 принятого текста.
- Контакты кандидата видны только после принятия приглашения или собственного отклика и только при действующем согласии. Каждый просмотр — запись в журнале доступа.
- Отзыв согласия закрывает контакты сразу, через 30 дней задача стирает имя, контакты, ФСП ID и навыки (ч. 5 ст. 21). Удаление аккаунта стирает их немедленно.
- При эксплуатации персональные данные граждан РФ ведутся в базах на территории РФ (ч. 5 ст. 18). Демонстрационный стенд запускается там, где его подняли, и эту локализацию не обещает.
- Неправомерная или случайная передача персональных данных: уведомление Роскомнадзора в течение 24 часов и результаты внутреннего расследования в течение 72 часов (ч. 3.1 ст. 21, редакция от 24.06.2025). Стенд такое уведомление не отправляет.
- Если модель включена, резюме уходит к ней без имени, почты, телефона и Telegram. Разбор запроса работодателя идёт правилами, без модели. Черновик задания запрашивается только когда задана вторая, другая модель: в запрос входят специализация, тема и уровень, без резюме. Черновик

сохраняется как принятый, лишь если обе модели назвали один ключ и этого ключа нет в тексте условия. В банк заданий и в сессию теста он не попадает.
- Несовершеннолетние скрыты из поиска и рейтинга без согласия законного представителя.
- Пароли — bcrypt. Ключ подписи токенов уникален для установки. Пароль ФСП платформа не видит (OIDC). Коды подтверждения хранятся только в виде HMAC.
- Лимиты на вход, регистрацию, письма, приглашения и поиск. Если хранилище лимитов недоступно, вход и регистрация закрываются, а не пропускают запросы.
- Решение о найме принимает человек: платформа ранжирует и объясняет, но не отсеивает автоматически (ст. 16 152-ФЗ).
- Чужая потребность, чужое задание, чужая сессия теста и чужая переписка отвечают 404, как и отсутствующие (`test_access.py`, смоук двух работодателей). Те же таблицы закрыты политиками RLS. Соединение приложения после входа становится ролью `fsp_app`: суперпользователь эти политики не видит, даже если они принудительные. Работодатель не читает чужую потребность и ключи заданий. Секреты в дереве ищет gitleaks, зависимости — pip-audit и npm audit. CI подписывает SBOM ключом Sigstore (cosign sign-blob). Подписи образов в реестре нет.

Сверка с OWASP ASVS уровня 1 — не сертификат. Строка говорит, что уже есть в коде и чего нет.

| Тема | Есть | Нет |
|---|---|---|
| Вход | Пароли bcrypt, подтверждение почты кодом, лимит попыток. Если хранилище лимитов недоступно, вход закрывается | Второй фактор |
| Сессия | Access-токен на 60 минут. Refresh на 30 дней лежит в HttpOnly-cookie и при обмене получает новый номер поколения. Выход увеличивает этот номер, смена пароля и удаление аккаунта увеличивают эпоху. Вход через ФСП ID — PKCE S256. Keycloak realm `fsp` входит в Compose | Access-токен лежит в `localStorage` и до истечения срока выход его не гасит. Демо-вход ФСП ID остаётся страницей провайдера, пока `FSP_ID_MODE=mock` |
| Доступ к чужим объектам | Чужая потребность, задание, сессия теста и переписка отвечают так же, как отсутствующие. RLS на потребностях, приглашениях, сессиях и подтверждениях грейда. Соединение приложения становится ролью `fsp_app` без прав суперпользователя. Ключи заданий работодатель не читает | Миграции идут от владельца базы, он суперпользователь. Обход политик остаётся у сида, воркера и публичных страниц |
| Криптография и канал | Подпись удостоверений Ed25519, коды подтверждения — HMAC. Заголовки nginx: `X-Frame-Options`, `nosniff`, `Referrer-Policy`, CSP с `script-src 'self'` | HTTPS есть у публичного стенда за Caddy. Локальный стенд отвечает по HTTP |
| Данные | Отдельные согласия с версией и SHA-256, отзыв, удаление аккаунта, журнал просмотра контактов | Уведомление Роскомнадзора стенд не отправляет |
| Зависимости | gitleaks, pip-audit, npm audit в CI. SBOM Python-зависимостей — CycloneDX из `requirements.lock`. Тот же файл подписывает cosign (ключ Sigstore, без ключа в репозитории) | Подпись образов в реестре |

## 11. За рамками MVP

Концепции защиты от фиктивных вакансий и интеграции с ATS: TRUST_AND_ATS.md. Дорожная карта — в README.

Когда появятся настоящие отклики, веса подбора нужно перекалибровать на исходах: сейчас они зафиксированы до замера и не подгоняются под синтетику. Дрейф банка уже считается (z и CUSUM по ключу), возвращать ключ из карантина можно только после разбора. Обучение ранжированию на том, кто принял приглашение, имеет смысл только на этих исходах. Выдача удостоверений по OID4VCI и вход через ЕСИА ждут контуров ФСП и государства: сейчас удостоверение подписывается `eddsa-jcs-2022`, а вход идёт по OIDC.

На симуляции `python -m app.validation.assessment --n 2000 --seed 7` порог по слабой теме снижает ложные сдачи с 7,7 % до 3,0 % и роняет точность решения с 78,5 % до 55,4 %, поэтому в решении о грейде его нет. Пересчёт различимости заданий увеличивает среднюю ошибку параметра с 0,146 до 0,283, поэтому в банк заданий он не пишется.

# Соответствие техническому заданию

Каждое требование ТЗ — где его увидеть в интерфейсе и где оно реализовано в коде. Экраны указаны по пунктам меню: «Кандидат → Профиль» значит раздел «Профиль» в кабинете кандидата. Демо-входы — в README.

Статусы: ✅ сделано · ◐ сделано частично, остаток указан · ◇ концепция (ТЗ просит концепцию, а не код).

## Раздел 2. Описание задачи

| Требование | Статус | Где увидеть | Где в коде |
|---|---|---|---|
| Обратная механика: работодатель сам находит категорию и выходит на кандидата с предложением и зарплатой | ✅ | Работодатель → Потребности → подборка → «Пригласить» | `app/api/v1/needs.py`, `offers.py` |
| Категория определяется опросом, выбором грейда и тестом, а не резюме | ✅ | Кандидат → Обзор → опрос; Тест на грейд | `app/services/survey.py`, `categorization.py`, `assessment.py` |
| Категория = специализация × грейд, справочник специализаций и грейдов | ✅ | Работодатель → Поиск, фильтры | `app/services/survey.py` (11 специализаций, 5 грейдов) |
| Внутри категории выше те, у кого сильнее подтверждённый профиль и есть опыт ФСП | ✅ | Подборка: полосы факторов, «Почему в выдаче» | `app/services/matching.py` |
| Тест устойчив к распространению заданий и даёт сопоставимую сложность | ✅ | Тест на грейд: новые параметры у каждого | `app/items/`, `app/services/cat.py`, VALIDATION_ASSESSMENT.md |
| Несколько описаний вакансий, на их основе система формирует задания | ✅ | Кандидат → Вакансии (9 вакансий); Работодатель → Потребность → «Как проверим», Вакансии → «Какой тест получится» | `app/services/vacancy_exam.py`, `seed/generate_seed.py` |
| Грейд не понижается, можно пройти тест ниже или выше, смена грейда ограничена по времени | ✅ | Тест на грейд: выбор уровня, срок до пересдачи, рекомендация теста на ступень выше | `app/services/assessment.py` (`decide_grade`, `retake_available_at`: 90 и 7 дней, тест ниже после провала — сразу) |
| Регулярные короткие задания от работодателя | ✅ | Работодатель → Задания или «Дать задание» в карточке; Кандидат → Задания. Плюс «Задача недели», собранная платформой из вакансии | `app/services/tasks.py`, `app/api/v1/tasks.py`; задача недели — `app/services/practice.py`, `offer_weekly_tasks` |
| Сценарий контакта, отличный от отклика (ориентир LHH) | ✅ | Работодатель → Приглашения; Кандидат → Приглашения | `app/api/v1/offers.py`, `candidates.py` |
| Связь профиля с ID участника ФСП, совместимость с Keycloak | ✅ | Вход → «Войти через ФСП ID» (единая точка входа); Кандидат → Профиль → привязка | `app/services/fsp/identity.py`, `app/api/v1/auth.py` (`/fsp/*`), FSP_INTEGRATION.md |
| Команда сама решает, какие данные ФСП получать и как показывать | ✅ | Кандидат → Результаты ФСП; карточка в выдаче | `app/services/fsp/registry.py`, `scoring/engine.py` |
| Корректная обработка кандидата без истории ФСП | ✅ | Вход `nofsp@demo.talent` | `matching.py`: ось выпадает без штрафа |

| Требование | Статус | Где увидеть | Где в коде |
|---|---|---|---|
| Собственная процедура оценки решения и её результаты | ✅ | — | SOLUTION.md, гл. 5 |
| Защита от фиктивных вакансий и недобросовестных работодателей (плюс) | ◐ | Компания: ИНН; приглашение: профиль компании | Работает: ИНН, вилка, лимиты, срок. Концепция: TRUST_AND_ATS.md |
| Интеграция с ATS (плюс) | ◇ | — | TRUST_AND_ATS.md |

## Раздел 2.1. Кабинет кандидата

| Требование | Статус | Где увидеть | Где в коде |
|---|---|---|---|
| Профиль вручную или загрузка PDF с распознаванием (ФИО, контакты, стек, грейд, роли, стаж, софт-скиллы) | ✅ | Профиль → «Загрузить PDF»: поля с цитатой и кнопка «Применить» | `app/services/pdf_profile.py`, `ai/assistant.py` |
| Автогенерация стандартизированного PDF-профиля | ✅ | Профиль → «Скачать PDF профиля» | `GET /candidates/me/profile.pdf` |
| Опрос, выбор грейда, тест, автоматическая категория | ✅ | Обзор → опрос; Тест на грейд | `PUT /candidates/me/survey`, `/assessment/*` |
| Регулярные задания влияют на актуальность профиля | ✅ | Задания, Обзор → «Задача недели»; у работодателя строки «Задание работодателя «...»: зачтено, вчера» и «Короткая практика ... N дн. назад» | `app/services/tasks.py` (`card_signal`), `app/services/search/pipeline.py` |
| Категория и грейд, история опроса и теста, тест ниже или выше с ограничением частоты | ✅ | Обзор; Тест на грейд → история | `GET /assessment/history`, `/assessment/cooldown` |
| Приглашения с вакансией и зарплатой, принять или отклонить | ✅ | Приглашения; после принятия — «Переписка» и «Закрыть контакты для этой компании» | `POST /candidates/me/offers/{id}/respond`, `/revoke-contacts`, `/offers/{id}/messages` |
| Вакансии, самостоятельный отклик, свои отклики и статусы | ✅ | Вакансии → «Мои отклики» | `app/api/v1/vacancies.py` |
| Настройки: приватность, согласия, связь с ФСП ID | ✅ | Профиль → видимость, согласия, ФСП ID, удаление аккаунта; Приглашения → закрыть контакты для одной компании | `PATCH /candidates/me/visibility`, `/me/consents`, `/me/offers/{id}/revoke-contacts` |
| Кандидат видит только свои приглашения и отклики | ✅ | — | проверка владельца в каждом методе |

## Раздел 2.1. Кабинет работодателя

| Требование | Статус | Где увидеть | Где в коде |
|---|---|---|---|
| Профиль компании: описание, направление, контакты | ✅ | Компания | `GET/PATCH /employer/me` |

| Требование | Статус | Где увидеть | Где в коде |
|---|---|---|---|
| Описание потребности: специалисты, чем занимается команда, стек, специализация, грейд | ✅ | Потребности → создать | `app/api/v1/needs.py` |
| Рекомендованные категории и кандидаты, обоснование попадания | ✅ | Потребность → «Где искать», подборка, «Почему в выдаче» | `recommended_categories`, `matching.score` |
| Поиск по всей базе: специализация, грейд, категория, стек, достижения ФСП | ✅ | Поиск | `GET /search/candidates` |
| Приглашение с описанием и зарплатой без обязательной вакансии, статусы | ✅ | «Пригласить» в подборке и поиске; Приглашения | `POST /offers/` |
| Публикация вакансий, редактирование, отклики и работа с ними | ✅ | Вакансии | `PUT /vacancies/{id}`, `PATCH /vacancies/applications/{id}/status` |
| Контакты закрыты до принятия приглашения или отклика | ✅ | Приглашения: контакты появляются после «Принять»; если кандидат закрыл контакты для компании — 403 `contacts_revoked` | `GET /offers/{id}/contacts`, журнал `access_logs` |
| Регулярные задания кандидатам и проверка ответов | ✅ | Задания → «Новое задание», ответы, «Зачтено / не зачтено»; «Дать задание» в поиске и подборке | `app/api/v1/tasks.py` |
| Переписка с кандидатом после принятия приглашения | ✅ | Приглашения → «Переписка» | `app/api/v1/messages.py` |

## Раздел 2.1. Поиск и подбор

| Требование | Статус | Где в коде |
|---|---|---|
| Выдача по категориям из опроса и теста, а не по резюме | ✅ | `category_assignments`, `matching.py` |
| Ранжирование по силе подтверждённого профиля и соответствию потребности, краткая объяснимость | ✅ | `matching.py`: факторы с весами и строки «плюс / минус / инфо» |
| Фильтрация и уточнение без потери подборки | ✅ | версии потребности `need_versions`, отобранные `need_shortlist` помнят версию |

## Раздел 2.2. Данные и правила видимости

| Правило | Статус | Где в коде |
|---|---|---|
| 1. Зарплата «от–до» в рублях обязательна в вакансии и приглашении | ✅ | `OfferCreate`, `VacancyIn`: обе границы обязательны, «до» ≥ «от» |
| 2. Приглашение содержит описание, зарплату, компанию, способ связи; контакты после согласия | ✅ | `offers.py`, `candidates.py` (`/me/offers`) |
| 3. Статусы приглашений и откликов видны обеим сторонам | ✅ | `offers.status`, `applications.status`, уведомления |
| 4. Регистрация по email с подтверждением адреса | ✅ | `app/services/identity.py`: код из 6 цифр или ссылка |

## Ответы постановщика на Q&A и как они учтены

| Ответ постановщика | Как сделано | Где в коде |
|---|---|---|
| Грейд определяется только тестом категории | Грейд даёт только основной тест специализации. Тест по вакансии, задачи недели, задания работодателя и результаты ФСП грейд не меняют | `app/services/assessment.py`, `is_vacancy_session` |
| Неподтверждённый грейд показывается ниже | До теста грейд помечен как заявленный и в оценке соответствия весит 0,85 от подтверждённого; в карточке строка «указан кандидатом, тестом не подтверждён» | `app/services/matching.py` |
| Повышение грейда — отдельным тестом | Тест на уровень подтверждает только этот уровень. Уверенный результат выше (от 90 % за следующий порог) даёт рекомендацию пройти тест на ступень выше, но грейд сам не растёт | `app/services/cat.py` (`RECOMMEND_NEXT_P`), `assessment.py` |
| Ограничение по времени, но тест ниже без потери попытки | Не сдали — тест на уровень ниже доступен сразу. Пересдача того же уровня — через 7 дней, прерванная попытка засчитывается. После смены грейда следующее повышение — через 90 дней | `retake_available_at`, `GRADE_COOLDOWN_DAYS`, `RETAKE_INTERVAL_DAYS` |
| Тест устойчив к утечке банка | Параметрические семейства вместо банка, вводимый ответ, карантин утёкших ключей, вопросы на понимание не повторяются во второй попытке больше чем на 40 % | `app/items/`, `item_monitor.py`, VALIDATION_ASSESSMENT.md |
| Языковая модель — российская, без требований к GPU | GigaChat, только для разбора резюме: каждый предложенный навык должен совпасть с дословной цитатой из текста, иначе отбрасывается. По умолчанию выключен (`GIGACHAT_CREDENTIALS` пуст), работает детерминированный разбор. Тест и подбор модель не используют, GPU не нужен | `app/services/ai/assistant.py` |
| Вакансии не импортируем | Вакансии создают только работодатели на платформе, демо-вакансии вымышлены | `app/api/v1/vacancies.py`, `seed/` |
| Регулярные задания пишет работодатель, проверяет система или работодатель | Задание с эталоном (число, выбор) проверяет система, открытое (скрипт, бизнес-решение) — работодатель. Задание для всех кандидатов специализации или одному кандидату | `app/services/tasks.py`, `app/api/v1/tasks.py` |
| ФСП ID — единая точка входа, вход можно имитировать | «Войти через ФСП ID» на входе и регистрации: OIDC с PKCE, демо-провайдер вместо Keycloak. Новый человек проходит экран согласий, аккаунт создаётся из реестра. Регистрация по email осталась | `app/api/v1/auth.py`, FSP_INTEGRATION.md |
| Кандидат может закрыть контакты | Глобальный отзыв согласия и закрытие контактов для одной компании: компания получает 403 `contacts_revoked`, событие пишется в журналы | `candidates.py` (`/revoke-contacts`), TRUST_AND_ATS.md |
| Нужна связь после принятия приглашения | Переписка по каждому принятому приглашению, после закрытия контактов — только чтение | `app/api/v1/messages.py` |

| Ответ постановщика | Как сделано | Где в коде |
|---|---|---|
| Достижения ФСП: соревнования не ранжировать по статусу, учитывать результат и количество | Методика `v3.1.0`: класс результата по месту (победитель, призёр, топ-10 %, верхняя половина, участие с весом 0), статус не учитывается. В сумму входят пять сильнейших событий. Количество результатов работает через байесовское сжатие: участия не поднимают среднее | `app/services/scoring/engine.py`, README |
| Несовершеннолетние участники | Скрыты из поиска и рейтинга без согласия законного представителя | `app/services/profile.py` (`adult_cutoff`) |
| Зарплата — до вычета НДФЛ | Поля вилки названы «Зарплата до вычета НДФЛ, ₽ в месяц», в API — `salary_from_gross`, `salary_to_gross` | `InviteDialog.tsx`, `app/models/offer.py` |
| Ресурсы стенда | Обычная машина: 8 контейнеров, около 1,5 ГБ памяти, без GPU | INSTALL.md |
| «Отрасль» в ТЗ — направление ИТ | Шаг опроса называется «Направление и специализация», поле предметной области необязательно | `frontend/src/components/SurveyCard.tsx`, `app/services/survey.py` |

## Раздел 3. Программные и архитектурные требования

| Требование | Статус | Где |
|---|---|---|
| 3.1 Обычная машина или недорогой сервер | ✅ | 8 контейнеров, около 1,5 ГБ памяти (INSTALL.md) |
| 3.2 ML и NLP для резюме, категоризации и подбора — преимущество | ✅ | Разбор резюме с цитатами, IRT-модель теста, ранжирование, GigaChat и эмбеддинги по флагу |
| 3.3 Ссылка на стенд или видео и инструкция локального запуска | ✅ | INSTALL.md, deploy/README.md |
| 3.3 Сквозной сценарий и объяснение механик | ✅ | README: сквозной сценарий |
| 3.4 Сопроводительная документация по восьми пунктам | ✅ | SOLUTION.md |
| 3.5 Открытый код и открытые библиотеки | ✅ | LICENSE (MIT), таблица лицензий в SOLUTION.md, гл. 8 |
| 3.5 Frontend, Backend и слой хранения разделены | ✅ | `frontend/`, `backend/`, PostgreSQL |
| 3.5 Docker, Dockerfile, docker-compose.yml | ✅ | `docker-compose.yml`, `backend/Dockerfile`, `frontend/Dockerfile`, проверка в CI |
| 3.5 OpenAPI: запросы, ответы, коды ошибок | ✅ | `/docs`, `/openapi.json`, API_ERRORS.md |
| 3.5 152-ФЗ, согласия, роли, защита паролей и токенов | ✅ | SOLUTION.md, гл. 10 |
| 3.5 Совместимость с Keycloak | ✅ | OIDC-клиент, FSP_INTEGRATION.md |
| 3.5 Кроссплатформенность, адаптивная вёрстка | ✅ | Все экраны от 390 px, светлая и тёмная темы (UI_AUDIT_2026-10-06.md) |
| 3.5 Десятки пользователей одновременно | ✅ | 40 пользователей, 0 ошибок (LOAD_TEST.md) |
| 3.5 Комментарии к сложным решениям, код соответствует документации | ✅ | Докстринги модулей теста, подбора и ФСП; цифры документации сверены с выводом команд |

## Раздел 5.2. Финальная сдача

| Что сдаётся | Где |
|---|---|
| Репозиторий с кодом | этот репозиторий |
| Сопроводительная документация в doc или pdf | docs/FSP_Talent_documentation.pdf — сборка всех документов, `make docs-pdf` |

| Что сдаётся | Где |
| --- | --- |
| Инструкция локального запуска | INSTALL.md |

# Сборка, развёртывание и локальный запуск

Инструкция для эксперта, который разворачивает решение на своей машине (ТЗ 3.3, 3.4). Один и тот же путь каждый раз проходит в CI: задание `compose-e2e` выполняет `make demo` и `make check` на чистой машине.

Время первого запуска — 5–10 минут: столько уходит на сборку образов. Повторный запуск занимает меньше минуты.

## 1. Что понадобится

| Что | Версия | Как проверить |
|---|---|---|
| Docker Engine или Docker Desktop | 24 и новее | `docker --version` |
| Docker Compose v2 | 2.17 и новее (нужен флаг `--wait`) | `docker compose version` |
| Git | любая | `git --version` |
| Свободная память | 4 ГБ | всё вместе занимает около 1,5 ГБ |
| Место на диске | 5 ГБ | образы и зависимости |
| Свободные порты | 80 и 8025 | порт 80 можно поменять, см. раздел 9 |

`make` нужен только для коротких команд. В Windows его обычно нет, поэтому для PowerShell рядом даны те же команды без `make`.

Подходит Linux, macOS (Intel и Apple Silicon) и Windows 10/11 с Docker Desktop на WSL 2. Python и Node.js на машине не нужны: всё собирается внутри контейнеров.

## 2. Получить код

```
git clone https://github.com/KonkovDV/fsp-talent.git
cd fsp-talent
```

Если код пришёл архивом, распакуйте его и откройте терминал в папке `fsp-talent`.

## 3. Запустить одной командой

**Linux и macOS**

```
make demo
```

**Windows (PowerShell)**

```
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
docker compose up -d --build --wait
docker compose exec -T -u appuser backend python -m seed.generate_seed
```

`make demo` выполняет те же три шага:

1. Создаёт `.env` из `.env.example`, если файла ещё нет. Править его не нужно. Ключ подписи токенов контейнер сам создаст при первом старте и сохранит в томе `vc_keys`.
2. Собирает образы, поднимает контейнеры и ждёт, пока контейнеры с проверкой здоровья (база, backend, worker, beat, frontend, Mailpit) станут `healthy`. Миграции базы применяются при старте backend.
3. Загружает демо-данные. Повторный запуск безопасен: уже загруженное не дублируется.

В конце команда печатает адреса и демо-входы.

## 4. Проверить, что всё работает

| Адрес | Что там |
|---|---|
| http://localhost | Сайт: кабинеты кандидата и работодателя |
| http://localhost/docs | Swagger UI: все методы API с запросами, ответами и кодами ошибок |
| http://localhost/openapi.json | Спецификация OpenAPI 3.1 |
| http://localhost/health | `{"status":"ok",...}` |
| http://localhost:8025 | Mailpit: письма с кодами подтверждения и уведомлениями |

Команда `docker compose ps` должна показать девять контейнеров в состоянии `running`. У всех, кроме `redis` и `nginx`, дополнительно стоит `healthy`.

Сквозная проверка API на запущенном стенде:

```
make check
```

Команда регистрирует нового кандидата (код подтверждения она берёт из Mailpit), привязывает ФСП ID через демо-провайдера, проходит адаптивный тест, ищет кандидата, отправляет приглашение и проверяет раскрытие контактов. Она же проверяет задания от работодателя (с проверкой системой и работодателем), вход через ФСП ID привязанного и нового кандидата, переписку по принятому приглашению и закрытие контактов для одной компании. Успешный прогон заканчивается строкой `SMOKE OK`.

> `make check` меняет демо-данные: например, подтверждает тестом грейд кандидата `fsp001@demo.talent`. Чтобы вернуть стенд к исходному виду, выполните `make reset` и снова `make demo`.

## 5. Демо-входы

Пароль у всех демо-аккаунтов: `demo12345`. Компании и люди вымышленные.

| Роль | Email | Что показывает |
|---|---|---|
| Кандидат | `fsp001@demo.talent` | Привязан ФСП ID, результаты из протоколов, ожидающие приглашения с зарплатой, задача недели |
| Кандидат | `fsp002@demo.talent` | Специализация «информационная безопасность», командные результаты ФСП |
| Кандидат | `nofsp@demo.talent` | Без истории ФСП: категория строится только по опросу и тесту |
| Работодатель | `hr@sibcode.demo` | Три потребности с подборками, приглашения во всех статусах, вакансии и отклики, три задания для кандидатов (одно открытое ждёт проверки) |

Чтобы показать регистрацию с нуля, заведите новый аккаунт на http://localhost/register. Код подтверждения придёт в Mailpit. Для привязки ФСП ID в демо-реестре свободны номера FSP2000–FSP2099, пароль `test`. Войти без регистрации можно кнопкой «Войти через ФСП ID» на странице входа: `FSP001` / `test` открывает аккаунт `fsp001@demo.talent`, свободный номер из FSP2000–FSP2099 / `test` создаёт новый аккаунт после экрана согласий.

Пошаговый сценарий показа — в README.

### Сценарий на три минуты

Ролик записывает автор. Стенд уже поднят (`make demo`), почта открыта на http://localhost:8025. Пароль демо-аккаунтов `demo12345`.

1. Регистрация на http://localhost/register: новый email, пароль, отметка «Мне исполнилось 18 лет», согласия. Код из письма в Mailpit.

2. Опрос: специализация, сфера, цель, ожидания по зарплате. На обзоре появляется категория, грейд помечен «не подтверждён».
3. Пробный тест: шесть заданий, без паузы. На экране итога написано, что грейд не подтверждается и в поиске не учитывается.
4. Вход работодателем `hr@sibcode.demo`. Поиск по специализации нового кандидата: карточка без имени, с объяснением места.
5. «Пригласить»: должность и вилка. Если вилка ниже ожиданий, видна подсказка, отправка не блокируется.
6. Кандидат открывает приглашение и принимает его. У работодателя появляются имя и контакты. До принятия их нет ни в карточке, ни в ответе API.

## 6. Тесты и проверки

Все команды запускаются на поднятом стенде, внутри контейнера backend.

| Что | Linux, macOS | Windows (PowerShell) |
|---|---|---|
| Юнит-тесты | `make test` | `docker compose exec -T backend python -m pytest tests/unit -q` |
| Стиль кода | `make lint` | `docker compose exec -T backend sh -c "pip install -q ruff==0.16.10 && ruff check app seed tests"` |
| Проверка теста и подбора на синтетике | `make validate` | `docker compose exec -T backend python -m app.validation.assessment --n 300 --seed 7 --check`, затем `docker compose exec -T backend python -m app.validation.matching` |
| Сквозная проверка API | `make check` | `docker compose exec -T -e SMOKE_BASE=http://localhost:8000/api/v1 -e MAILPIT_URL=http://mailpit:8025 backend python tests/smoke_api.py` |
| Нагрузка: 40 пользователей в течение минуты | `make load` | `docker compose exec -T -e SMOKE_BASE=http://localhost:8000/api/v1 -e MAILPIT_URL=http://mailpit:8025 backend python tests/load_api.py` |

Внутри контейнера нет Node.js, Go, Java, Kotlin и .NET, поэтому тесты, которые перезапускают задания на этих языках, пропускаются с пометкой `skipped`. В CI все языки установлены и эти тесты выполняются.

Полные цифры из документации воспроизводятся так:

```
docker compose exec -T backend python -m app.validation.assessment --n 2000 --seed 7
docker compose exec -T backend python -m app.validation.matching
docker compose exec -T backend python -m app.validation.matching --seed 7
```

## 7. Остановить, сбросить, обновить

| Действие | Команда |
|---|---|
| Остановить, данные сохраняются | `make down` или `docker compose down` |
| Запустить снова | `make up` или `docker compose up -d --wait` |
| Удалить все данные (база, ключи) | `make reset` или `docker compose down -v` |
| Обновить код и пересобрать | `git pull`, затем `make demo` |
| Журнал API | `make logs` или `docker compose logs -f backend` |
| Консоль базы | `docker compose exec postgres psql -U fsp fsp_talent` |

## 8. Что входит в стенд

| Контейнер | Образ | Назначение |
|---|---|---|
| `nginx` | nginx:alpine | Единая точка входа: сайт, `/api`, `/docs`, заголовки безопасности (CSP) |

| Контейнер | Образ | Назначение |
|---|---|---|
| `frontend` | собирается из `frontend/` (node:20 → nginx) | React-приложение для обеих ролей |
| `backend` | собирается из `backend/` (python:3.12-slim) | FastAPI, миграции Alembic при старте, 2 процесса uvicorn |
| `worker` | тот же образ | Celery: пересчёт профилей, истечение приглашений, задача недели, удаление данных по 152-ФЗ |
| `beat` | тот же образ | Расписание задач Celery |
| `postgres` | pgvector/pgvector:pg16 | PostgreSQL 16 с pgvector и pg_trgm |
| `redis` | redis:7.2-alpine | Лимиты запросов, состояние входа через ФСП ID, брокер Celery |
| `mailpit` | axllent/mailpit | Почтовый ящик для писем на локальном стенде |

Наружу открыт только порт nginx. API (8010), отдельный frontend (3010) и Mailpit (8025) слушают только 127.0.0.1. У базы и Redis портов на хосте нет.

## 9. Настройки `.env`

Для демо `.env` можно не трогать. Основные параметры:

| Переменная | По умолчанию | Зачем |
|---|---|---|
| `SECRET_KEY` | заглушка | Ключ подписи токенов. Заглушку контейнер заменит случайным ключом. На публичном стенде задайте свой: `openssl rand -hex 32` |
| `HTTP_PORT` | 80 | Порт сайта на хосте. Если 80 занят: `HTTP_PORT=8080` и `PUBLIC_BASE_URL=http://localhost:8080` |
| `PUBLIC_BASE_URL` | http://localhost | Адрес, из которого строятся ссылки в письмах |
| `FSP_MODE` | mock | `mock` — тестовый реестр ФСП, `real` — HTTP-клиент по контракту из FSP_INTEGRATION.md |
| `FSP_ID_MODE` | mock | `mock` — демо-страница провайдера, `oidc` — Keycloak Федерации (`FSP_ID_ISSUER`, `FSP_ID_CLIENT_ID`, `FSP_ID_CLIENT_SECRET`) |
| `AI_ENABLED`, `GIGACHAT_CREDENTIALS` | true, пусто | Без ключа GigaChat резюме разбирается правилами. С ключом модель только добавляет навыки с дословной цитатой |
| `EMBEDDINGS_MODE` | off | `model` включает смысловой поиск по эмбеддингам. Нужны зависимости `.[ai]` (sentence-transformers и torch), в демо-образ они не входят |
| `MAIL_MODE`, `SMTP_*` | smtp → Mailpit | Настоящий SMTP на стенде |

## 10. Запуск без Docker (для разработки)

Нужны Python 3.12, Node.js 20+, PostgreSQL 16 с расширением pgvector и Redis 7.

```
# backend
cd backend
python -m venv .venv && source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -c requirements.lock -e ".[test]"
export DATABASE_URL=postgresql+asyncpg://fsp:fsp@localhost:5432/fsp_talent
export REDIS_URL=redis://localhost:6379/0
export SECRET_KEY=$(python -c "import secrets; print(secrets.token_hex(32))")
export MAIL_MODE=console VC_PRIVATE_KEY_PATH=./vc_ed25519.pem
alembic upgrade head
python -m seed.generate_seed
uvicorn app.main:app --reload --port 8000

# frontend, в другом терминале
cd frontend
npm ci
npm run dev        # http://localhost:5173, запросы /api уходят на :8000
```

Коды подтверждения при `MAIL_MODE=console` печатаются в журнал uvicorn.

## 11. Публичный стенд с HTTPS

Продовая сборка (`deploy/docker-compose.prod.yml`) использует Caddy с автоматическим сертификатом Let's Encrypt и ночные резервные копии базы. Домен покупать не нужно: подходит адрес вида `203-0-113-10.sslip.io`. Шаги описаны в deploy/README.md.

## 12. Если что-то пошло не так

| Симптом | Причина и решение |
|---|---|
| `port is already allocated` для 80 | Порт занят другим сервером. Задайте в `.env` `HTTP_PORT=8080` и `PUBLIC_BASE_URL=http://localhost:8080`, затем `make demo` |
| `unknown flag: --wait` | Compose старше 2.17. Обновите Docker Desktop или запустите без флага и подождите минуту: `docker compose up -d --build` |
| `env file .env not found` | Не создан `.env`. Выполните `cp .env.example .env` (Windows: `Copy-Item .env.example .env`) |
| `backend` не становится `healthy` | Посмотрите `docker compose logs backend`. Если там `SECRET_KEY is a published placeholder`, том `vc_keys` недоступен на запись: задайте `SECRET_KEY` в `.env` |
| Вход не работает, `503` | Redis не запущен: `docker compose ps redis`, затем `docker compose up -d redis` |
| Письмо с кодом не пришло | Откройте Mailpit http://localhost:8025. Код можно запросить повторно через минуту |
| После `git pull` сайт показывает старую версию | Пересоберите образы: `make demo` (или `docker compose up -d --build --wait`) |
| Нужен чистый стенд | `make reset`, затем `make demo` |
| Сборка падает на `npm ci` или `pip install` | Нет доступа к npm или PyPI. Проверьте прокси Docker Desktop (Settings → Resources → Proxies) |

# Проверка теста на грейд

ТЗ (раздел 2) требует тест, устойчивый к утечке: без единого набора вопросов и без вызова языковой модели на каждого кандидата. Там же названы устойчивость повторной сдачи и различающая сила заданий. Все три свойства проверены симуляцией через те же функции, что работают в API (`app/services/cat.py`, `app/services/item_monitor.py`), так что цифры ниже описывают именно рабочий код.

## Как устроен тест

- **Задания — параметрические семейства** (`backend/app/items/families/`): 17 семейств с вычисляемым ответом (Python, SQL, JavaScript, алгоритмы, графы, сети, безопасность, ML-метрики, встраиваемые системы, границы и расхождения тестов, метрики и доли аналитика, непокрытые требования и шаги цепочки, пакеты и опоздания в конвейере данных) и 2 концептуальных пула. У каждого семейства 5 уровней сложности. Правильный ответ получается запуском фрагмента кода, SQL-запроса или формулы; юнит-тест `test_item_families.py` перезапускает каждый сгенерированный фрагмент отдельно — в Python, Node.js и SQLite — и сверяет ответ.
- **Числовой ответ вводится.** Если у задания с кодом правильный ответ — целое число, кандидат печатает его сам: вариантов на экране нет. Так устроены Python, SQL, подсчёт операций, графы, границы тестов, метрики аналитика и часть заданий по JavaScript, сетям и встраиваемым системам: в симуляции ниже это 79 % выданных заданий. Ответы вроде шестнадцатеричной строки, адреса или десятичной дроби остаются выбором из четырёх: их формат вводить неоднозначно.
- **Никакого смешения специализаций**: тест backend собирается только из семейств backend по плану (Python 30 %, SQL 25 %, алгоритмы 20 %, …). Концептуальных вопросов не больше 30 %.
- **IRT 3PL**: $P = c + (1 - c)\cdot\sigma(a(\theta - b))$. Угадывание $c = 0{,}02$ для вводимого числа и $c = 1/4$ для четырёх вариантов. Оценка способности $\theta$ считается как среднее апостериорного распределения на сетке (EAP), априорное распределение центрировано на пороге заявленного грейда. Селектор ещё не знает, какой экземпляр выпадет, поэтому берёт ожидаемое с семейства на этом уровне (`expected_guess`): долю вводимых ответов считает по 24 сгенерированным заданиям.
- **Почему это важно на пороге.** Информация задания 3PL при $\theta = b$ и $a = 1$ равна $(P - c)^2(1 - P) / ((1 - c)^2 P)$, где $P = (1 + c) / 2$. При $c = 0{,}25$ это 0,15, при $c = 0$ это 0,25: четыре варианта оставляют 60 % информации задания ровно на границе грейда.
- **Адаптивный выбор под решение о грейде**: задание выбирается по информации Фишера в точке порога, а не в текущей оценке $\theta$; среди пяти лучших — случайно; доли тем ограничены планом; слишком часто выдаваемые ключи получают штраф.
- **Остановка**: не меньше 8 и не больше 20 заданий. Тест заканчивается, когда апостериорная вероятность уровня выходит за 10 % или 90 % и вопрос о следующем уровне уже ясен. Раньше этого его может остановить обобщённое отношение правдоподобия (GLR) у порога: зона безразличия ±0,25, порог $\ln 9$ при ошибках 10 % с каждой стороны. Остановка случается только если отношение и апостериорное распределение смотрят в одну сторону. Пробный проход из 15 заданий и 5 минут останавливается по счётчику, без GLR. Среди пяти лучших заданий срабатывает Симпсон–Хеттер, когда по ключу уже есть доля показов.
- **Решение**: грейд подтверждён при $P(\theta \geq \text{порог}) \geq 75$ %. Тест на уровень подтверждает только этот уровень: при $P(\theta \geq \text{следующий порог}) \geq 90$ % сессия рекомендует отдельный тест на ступень выше, но грейд не повышает. Подтверждённый грейд не понижается; хранится отдельно для каждой специализации. Тест по вакансии и пробный проход из 15 заданий грейд не меняют. Грейд «стажёр» стоит на ступень ниже junior: порог $\theta = -2{,}5$, и под него выбираются задания первого уровня.
- **Навыки, подтверждённые тестом**: ≥ 75 % верных ответов по навыку на ≥ 3 заданиях; в профиле они отмечены `test_confirmed` и отличаются от заявленных.
- **Мониторинг утечки**: по каждому ключу «семейство:уровень» (и по каждому статичному вопросу) копится $z = (\text{верных} - \text{ожидаемых})/\text{sd}$, где ожидание считается по $\theta$, оценённой **без заданий того же семейства**: так утёкшее семейство не завышает $\theta$ и не прячет сигнал. Рядом копится односторонняя накопленная сумма избытка верных ответов. При $z \geq 3{,}5$ или при пересечении порога этой суммы на 30+ ответах ключ уходит в карантин: больше не выдаётся, а его ответы не учитываются. Статистика открыта: `GET /api/v1/public/assessment/items`.

- **Калибровка**: `python -m app.validation.calibrate` пересчитывает сдвиги трудности по реальным ответам (тоже без своего семейства) и обнуляет счётчики дрейфа.
- **Защита сессии**: правильный ответ не покидает сервер до конца теста; у вводимого задания варианты тоже остаются на сервере. Ответ принимается только на текущее задание по HMAC-токену; время засекает сервер (3 минуты на задание с кодом, 75 секунд на вопрос); прерванная или просроченная сессия считается попыткой; пересдача того же уровня — через 7 дней; после проваленного теста тест на уровень ниже доступен сразу.

## Симуляция

Кандидаты: θ ~ N(0,3; 1), специализация случайна среди всех одиннадцати направлений, включая инженерию данных. 60 % заявляют свой истинный грейд, 30 % — на ступень выше, 10 % — ниже. «Правда» специально не совпадает с моделью сервиса: у каждого ключа свой неизвестный сервису сдвиг трудности N(0; 0,3²) и множитель дискриминации U(0,8; 1,2). Задание вводимое с той долей, которую реально даёт его генератор. Перед мониторингом прогнана когорта для калибровки.

Сценарии «без утечки» и «все задания с вариантами» проходят одни и те же кандидаты, поэтому форматы ответа сравниваются напрямую. Сценарии с утечкой — отдельные когорты того же размера.

Симуляция: 2000 кандидатов на сценарий, зерно генератора 7. Команда: `python -m app.validation.assessment --n 2000 --seed 7`. Ответ вводится в 82 % выданных заданий.

| Сценарий | Точность «сдал/не сдал» | Точность вне ±0,5 от порога | Сдали из тех, кто на уровне | Ложно сдали | Ложная рекомендация теста выше | Заданий: среднее / 90 % | Решено до лимита | Макс. экспозиция ключа | Доля концептуальных |
|---|---|---|---|---|---|---|---|---|---|
| Без утечки | 78.5% | 91.5% | 73.0% | 7.7% | 1.0% | 14.3 / 20 | 63% | 34% | 7% |
| Без утечки, все задания с вариантами | 72.7% | 84.0% | 64.3% | 6.3% | 0.7% | 16.3 / 20 | 49% | 46% | 13% |
| Утечка у 30 %, без мониторинга | 78.8% | 90.8% | 77.8% | 18.4% | 2.9% | 14.1 / 20 | 64% | 35% | 7% |
| Утечка у 30 %, ключи на карантине | 78.6% | 91.7% | 75.6% | 13.4% | 1.9% | 14.5 / 20 | 61% | 35% | 8% |

Повторный тест: 1000 кандидатов проходят тест дважды, задания каждый раз новые.

| Формат | Тот же итог «сдал/не сдал» | Каппа Коэна | То же вне ±0,5 от порога | Тот же грейд | Корреляция оценок θ |
|---|---|---|---|---|---|
| Числовой ответ вводится | 76% | 0.51 | 85% | 63% | 0.85 |
| Все задания с вариантами | 70% | 0.41 | 77% | 57% | 0.84 |

Различающая сила заданий в сценарии «без утечки». r — точечно-бисериальная корреляция ответа с оценкой способности по остальным семействам теста.

| Формат ответа | Ключей с ≥ 30 ответами | Медиана r | Ключей с r ≥ 0,20 | Верно у тех, кто на 0,5 слабее задания | Верно у тех, кто на 0,5 сильнее |
|---|---|---|---|---|---|
| Число вводится | 52 | 0.22 | 62% | 29% | 72% |
| Выбор из четырёх | 16 | 0.15 | 19% | 55% | 80% |

- Мониторинг утечки: из 37 утёкших ключей с ≥ 30 ответами пойманы 18, ложных срабатываний среди остальных — 1; на чистых данных — 5 из 68 ключей.
- Случайное угадывание проходит тест в 0.0% случаев (все задания с вариантами — 0.0%).
- Точный грейд в сценарии «без утечки»: 57.0%, в пределах одной ступени: 84.0%, завышение: 2.2%.
- Взвешенная каппа повторного теста (квадратичные веса по грейду): 0.52.
- Ключ от другого экземпляра того же семейства: сдают 0.0%. Ключ от того же экземпляра (фиксированный банк): сдают 100.0%.
- Второй сигнал утечки (CUSUM): поймано 13 утёкших ключей, ложных тревог на чистых данных — 2. Карантин ставится, если сработал z или CUSUM.
- Калибровка различимости, средняя ошибка a: до 0.146, после 0.283. Банк заданий её не принимает: ошибка на симуляции не снижается.
- Если слабая тема блокирует сдачу: точность 55.4%, ложные сдачи 3.0% (в продукте порог по теме не включён, пока сравнение не подтвердит его).
- Точность классификации по Руднеру у порога заявленного грейда: 83.5%, согласованность 79.0%. Вероятность принятого решения по апостериорному распределению: 83.5%.
- Перекрытие заданий: в среднем другой кандидат этой же симуляции видел 15.2% заданий данного кандидата. Максимальная экспозиция ключа — в таблице выше.
- Экспозиция по регионам: четыре метки назначены отдельным генератором и в выбор задания не входят. Максимальная доля одного ключа в группах: 33%, 35%, 33%, 37%. Отношение меньшей к большей: 0.88.

Матрица «истинный грейд по θ × грейд, который назначила сессия» в сценарии «без утечки». Строка «ниже» — способность ниже стажёра или сессия не назначила грейд.

| Истинный \ назначенный | ниже | intern | junior | middle | senior | lead |
|---|---|---|---|---|---|---|
| ниже | 2 | 1 | 0 | 0 | 0 | 0 |
| intern | 24 | 36 | 1 | 0 | 0 | 0 |
| junior | 62 | 86 | 194 | 15 | 0 | 0 |
| middle | 113 | 1 | 180 | 429 | 19 | 0 |
| senior | 112 | 0 | 0 | 139 | 332 | 9 |
| lead | 8 | 0 | 0 | 0 | 90 | 147 |

Что показывают таблицы:

- **Ввод числа делает тест точнее и короче.** На одних и тех же 2000 людях точность вне ±0,5 от порога растёт с 84,0 % до 91,5 %, а доля сдавших среди тех, кто действительно на уровне, — с 64,3 % до 73,0 %. Тест укорачивается с 16,3 до 14,3 задания, а до лимита заканчиваются 63 % сессий вместо 49 %.
- **Повторная сдача устойчива.** Каппа Коэна учитывает совпадение «сдал / не сдал» сверх случайного: 0,51 против 0,41 у формата с вариантами. Вне ±0,5 от порога итог совпадает в 85 % пар против 77 %, корреляция двух оценок θ — 0,85 против 0,84.
- **Задания с вводимым числом хорошо различают уровни.** Медиана r = 0,22, и порог 0,20 проходят 62 % таких ключей. Кандидат на 0,5 слабее задания решает его в 29 % случаев, а на 0,5 сильнее — в 72 %. У выбора из четырёх медиана r = 0,15: слабый кандидат добирает до 55 % за счёт угадывания.
- **Решение надёжно там, где оно что-то значит.** Ширина полосы одного грейда — 1,0, и вне ±0,5 от порога тест прав в 91,5 % случаев. Кандидата в пределах ±0,3 от порога за 20 заданий нельзя уверенно отнести к одной стороне: это свойство самой задачи, а не конкретного теста.
- **Утечка не ломает тест.** Даже если 30 % кандидатов знают ответы главного семейства своей специализации, точность вне ±0,5 от порога остаётся 90,8 %. Карантин снижает долю ложно сдавших с 18,4 % до 13,4 % и находит 18 утёкших ключей из 37, с одним ложным срабатыванием на остальных. На чистых данных карантин отмечает 5 ключей из 68. Первая линия защиты — генерация: каждый кандидат получает новые экземпляры заданий.
- **Угадыванием тест не пройти**: доля случайных угадывателей, сдавших тест, — 0 %. Ложная рекомендация теста на ступень выше без утечки — 1,0 %: грейд при этом не меняется. Точный грейд совпадает в 57,0 % случаев, в пределах одной ступени — в 84,0 %, завышение — 2,2 %. Взвешенная каппа

повторного теста — 0,52. Точность классификации по Руднеру у порога заявленного грейда — 83,5 %, согласованность — 79,0 %. Другой кандидат той же симуляции видел в среднем 15,2 % заданий. Матрица выше показывает, что завышение больше чем на ступень почти не встречается.

- **Ключ от другого экземпляра того же семейства не проходит**: сдают 0 %. Ключ от того же экземпляра, как в заранее опубликованном банке, даёт 100 %. Второй сигнал утечки ловит 13 утёкших ключей и на чистых данных даёт 2 ложные тревоги. Вместе с z карантин отмечает 18 из 37.
- **Порог по теме и пересчёт различимости в банк не записаны.** Если слабая тема блокирует сдачу, ложные сдачи падают с 7,7 % до 3,0 %, но точность решения падает с 78,5 % до 55,4 %. Оценка различимости увеличивает среднюю ошибку a с 0,146 до 0,283.

Эти свойства закреплены в CI: при каждом изменении запускается `python -m app.validation.assessment --n 300 --seed 7 --check`, и сборка падает, если нарушен хотя бы один порог: точность вне ±0,5 не ниже 85 %, ложно сдавшие не больше 12 %, ложная рекомендация теста выше не больше 3 %, каппа пересдачи не ниже 0,35, случайное угадывание не больше 2 %, медиана r не ниже 0,20 для вводимого числа и 0,15 для выбора из четырёх, сдача с ключом от другого экземпляра того же семейства не больше 2 %.

Вводимый ответ сравнивается на сервере функцией `same_number`: она принимает лишние пробелы, знак плюс и типографский минус, так что формат записи не влияет на результат. Когда накопятся реальные ответы, команда `python -m app.validation.calibrate` уточнит трудность каждого ключа по ним.

Воспроизвести: `cd backend && python -m app.validation.assessment --n 2000 --seed 7`.

## Отдельный прогон на 5000

Эта таблица не заменяет прогон на 2000 и не входит в пороги CI. Команда: `python -m app.validation.assessment --n 5000 --seed 7`. Ответ вводится в 82% выданных заданий. Порог по теме и запись оценённой различимости по-прежнему выключены: на этих 5000 порог даёт точность 55.9% и ложные сдачи 4.3%, а оценка различимости увеличивает ошибку a с 0.145 до 0.267.

| Сценарий | Точность «сдал/не сдал» | Точность вне ±0,5 от порога | Сдали из тех, кто на уровне | Ложно сдали | Ложная рекомендация теста выше | Заданий: среднее / 90 % | Решено до лимита | Макс. экспозиция ключа | Доля концептуальных |
|---|---|---|---|---|---|---|---|---|---|
| Без утечки | 78.4% | 92.4% | 73.4% | 8.4% | 1.0% | 14.2 / 20 | 63% | 40% | 7% |
| Без утечки, все задания с вариантами | 70.9% | 85.1% | 63.3% | 9.0% | 0.8% | 16.4 / 20 | 49% | 47% | 13% |
| Утечка у 30 %, без мониторинга | 79.1% | 91.5% | 79.2% | 21.0% | 2.6% | 14.1 / 20 | 64% | 40% | 7% |
| Утечка у 30 %, ключи на карантине | 76.7% | 89.0% | 73.2% | 13.8% | 1.9% | 14.8 / 20 | 58% | 51% | 10% |

Повторный тест: 2500 кандидатов проходят тест дважды, задания каждый раз новые.

| Формат | Тот же итог «сдал/не сдал» | Каппа Коэна | То же вне ±0,5 от порога | Тот же грейд | Корреляция оценок θ |
|---|---|---|---|---|---|
| Числовой ответ вводится | 74% | 0.48 | 84% | 62% | 0.85 |
| Все задания с вариантами | 70% | 0.39 | 77% | 55% | 0.83 |

| Формат ответа | Ключей с ≥ 30 ответами | Медиана r | Ключей с r ≥ 0,20 | Верно у тех, кто на 0,5 слабее задания | Верно у тех, кто на 0,5 сильнее |
|---|---|---|---|---|---|
| Число вводится | 55 | 0.21 | 56% | 31% | 72% |
| Выбор из четырёх | 18 | 0.14 | 22% | 55% | 80% |

- Мониторинг утечки: из 38 утёкших ключей с ≥ 30 ответами пойманы 25, ложных срабатываний среди остальных — 1; на чистых данных — 12 из 73 ключей.
- Случайное угадывание проходит тест в 0.0% случаев (все задания с вариантами — 0.0%).
- Точный грейд в сценарии «без утечки»: 56.8%, в пределах одной ступени: 84.6%, завышение: 2.3%.
- Взвешенная каппа повторного теста (квадратичные веса по грейду): 0.53.
- Ключ от другого экземпляра того же семейства: сдают 0.0%. Ключ от того же экземпляра (фиксированный банк): сдают 100.0%.
- Второй сигнал утечки (CUSUM): поймано 15 утёкших ключей, ложных тревог на чистых данных — 2. Карантин ставится, если сработал z или CUSUM.
- Точность классификации по Руднеру у порога заявленного грейда: 83.6%, согласованность 79.3%. Вероятность принятого решения по апостериорному распределению: 83.6%.
- Перекрытие заданий: в среднем другой кандидат этой же симуляции видел 16.9% заданий данного кандидата.
- Экспозиция по регионам на этом прогоне: 42%, 40%, 40%, 37%, отношение меньшей к большей 0.89. В опубликованную проверку справедливости входит прогон на 2000.

| Истинный \ назначенный | ниже | intern | junior | middle | senior | lead |
|---|---|---|---|---|---|---|
| ниже | 11 | 1 | 0 | 0 | 0 | 0 |
| intern | 58 | 104 | 6 | 0 | 0 | 0 |
| junior | 123 | 209 | 506 | 24 | 0 | 0 |
| middle | 319 | 2 | 439 | 988 | 58 | 0 |
| senior | 245 | 0 | 0 | 405 | 856 | 28 |
| lead | 24 | 0 | 0 | 0 | 218 | 376 |

# Проверка подбора кандидатов

Механика подбора даёт 35 % оценки решения. Качество подбора проверено на синтетическом наборе со **скрытой правдой**: подбор видит только зашумлённые наблюдения, как в жизни, а релевантность считается по правде. Код проверки: `backend/app/validation/matching.py`. Ранжирует та же функция `app.services.matching.score`, что работает в API.

## Как устроена оценка соответствия

- Попадание в список решает соответствие потребности: навыки 0,50, грейд 0,20, специализация 0,20, текст 0,05, сфера 0,05.
- Навык засчитывается с весом доказательства: подтверждён тестом 1,0, цитатой из резюме 0,75, заявлен 0,55. Близкая технология (MySQL вместо PostgreSQL) даёт частичный зачёт 0,3.
- Грейд: точное совпадение 1,0, на ступень выше 0,6, на ступень ниже 0,35; грейд без подтверждения тестом умножается на 0,85.
- Среди кандидатов с одинаковым соответствием (до сотой) порядок задают тест (0,62) и протокол ФСП (0,38). Они сдвигают оценку меньше чем на 0,01 и не поднимают человека с худшим совпадением навыков.
- Неприменимый фактор не учитывается, веса остальных нормируются. Если в потребности нет сферы или у кандидата нет истории ФСП, этот фактор просто выпадает, и кандидат не теряет баллы (ТЗ 2.1).
- Каждый фактор оставляет в карточке строку «плюс / минус / инфо», работодатель видит и сами веса.
- Для потребности дополнительно подбираются **рекомендуемые категории**: ячейки «специализация × грейд» с самыми сильными совпадениями.

## Синтетический набор

Кандидат: специализация, способность θ ~ N(0, 1), реальные навыки. Наблюдения искажены так, как это бывает в анкетах: заявленные навыки теряют 25 % реальных и добавляют около 15 % ложных; 30 % заявленных навыков подтверждены цитатой; заявленный грейд у 30 % кандидатов на ступень выше реального и у 10 % ниже; тест проходят 45 %, он подтверждает реальные навыки и уровень; у 55 % есть история ФСП с LCB, связанной с θ.

Релевантность (0–3) считается только по правде: 3 — та же специализация, все обязательные навыки реальны, грейд совпадает; 2 — специализация та же или смежная, навыки реальны, грейд отличается не больше чем на ступень; 1 — реальна хотя бы половина навыков; 0 — иначе.

## Результаты

Специализации в наборе — все одиннадцать направлений продукта. Релевантность считается по скрытым навыкам и грейду и не смотрит на формулу оценки.

### Набор с зерном 2026

Команда: `python -m app.validation.matching`. 800 кандидатов, 78 потребностей с релевантными кандидатами, SHA-256 `c2d2ac57842c6264a07153923e99321dbd9eefa9538500dc55ca90e4af1891c8`.

| Ранжирование | nDCG@10 | P@10 (оценка ≥ 2) | MAP |
|---|---|---|---|
| Случайный порядок | 0.034 | 0.035 | 0.043 |
| Базовая линия: совпадение заявленных навыков | 0.615 | 0.565 | 0.548 |
| ФСП Талант: оценка соответствия | 0.661 | 0.663 | 0.618 |
| без весов доверия к навыкам | 0.655 | 0.678 | 0.606 |
| без теста | 0.661 | 0.668 | 0.625 |
| без оси ФСП | 0.661 | 0.664 | 0.619 |

| Ранжирование | nDCG@10 | P@10 (оценка ≥ 2) | MAP |
|---|---|---|---|
| без близких технологий | 0.659 | 0.663 | 0.613 |

### Набор с зерном 7

Команда: `python -m app.validation.matching --seed 7`. 800 кандидатов, 73 потребности с релевантными кандидатами, SHA-256 `4ffe49df166195e9ab2da714028140777f999b97dbafb08566d0d4c0ef7201e8`.

| Ранжирование | nDCG@10 | P@10 (оценка ≥ 2) | MAP |
|---|---|---|---|
| Случайный порядок | 0.041 | 0.034 | 0.040 |
| Базовая линия: совпадение заявленных навыков | 0.626 | 0.519 | 0.523 |
| ФСП Талант: оценка соответствия | 0.713 | 0.625 | 0.632 |
| без весов доверия к навыкам | 0.675 | 0.585 | 0.598 |
| без теста | 0.707 | 0.627 | 0.631 |
| без оси ФСП | 0.713 | 0.625 | 0.631 |
| без близких технологий | 0.704 | 0.610 | 0.625 |

## Выводы

- Подбор обходит поиск по ключевым словам на обоих наборах: nDCG@10 выше на 0,046 и 0,087, MAP — на 0,070 и 0,109.
- Веса доверия к навыкам на зерне 7 поднимают nDCG@10 с 0,675 до 0,713. На зерне 2026 разница 0,006: заявленные навыки уже хорошо разделяют этот набор.
- Тест и ось ФСП уточняют порядок среди одинакового соответствия и не обязаны поднимать nDCG. На зерне 2026 nDCG@10 равен 0,661 и с осью ФСП, и без неё. На зерне 7 обе цифры равны 0,713. Кандидат без истории ФСП из-за этого не теряет баллы. Ось ФСП в симуляции задана сразу нижней границей LCB, поэтому методика `v3.1.0` эти таблицы не переписывает. Фактор теста берёт нижнюю границу Уилсона по 12 ответам, округлённым из вероятности уровня: из-за этого полная оценка на зерне 2026 стала 0,661, на зерне 7 — 0,713.
- `--check` требует, чтобы полная оценка была выше и ключевых слов, и случайного порядка по nDCG@10. Эта проверка стоит в CI.
- Слепая разметка смотрит только на заявленный профиль и в одном случае из пяти сдвигает оценку на ступень. Против меток, посчитанных по скрытой правде, каппа Коэна 0,46 при зерне разметчика 2027. Зерно набора при этом 2026, контрольная сумма и nDCG не меняются, веса по каппе не подгоняются. На файле жюри скрытой правды нет, поэтому каппа там не считается.

## Разметка жюри

Своих пар у хакатона нет. Файл для жюри — JSONL, одна пара на строку: потребность, наблюдаемый профиль кандидата и оценка 0…3. Оценки независимы от формулы. Команда из каталога репозитория:

```
cd backend && python -m app.validation.matching --pairs ../docs/examples/jury_pairs.jsonl
```

Она печатает nDCG@10, P@10 и MAP. Три строки примера:

```
{"need": {"id": "n1", "role": "backend", "grade": "middle", "skills": ["Python", "PostgreSQL"]},
"candidate_id": "c1", "candidate": {"observed": {"role": "backend", "grade": "middle", "grade_source":
"assessed", "skills": {"Python": "test_confirmed", "PostgreSQL": "test_confirmed"}, "test_p": 0.9,
"fsp_lcb": 0.6, "industry": "fintech"}}, "label": 3}
{"need": {"id": "n1", "role": "backend", "grade": "middle", "skills": ["Python", "PostgreSQL"]},
"candidate_id": "c2", "candidate": {"observed": {"role": "backend", "grade": "junior", "grade_source":
"declared", "skills": {"Python": "self_reported"}, "test_p": null, "fsp_lcb": null, "industry":
"fintech"}}, "label": 1}
{"need": {"id": "n1", "role": "backend", "grade": "middle", "skills": ["Python", "PostgreSQL"]},
"candidate_id": "c3", "candidate": {"observed": {"role": "ml", "grade": "senior", "grade_source":
"declared", "skills": {"Python": "self_reported"}, "test_p": null, "fsp_lcb": 0.4, "industry":
"fintech"}}, "label": 0}
```


Тот же расчёт принимает JSON в формате `--export`: `python -m app.validation.matching --import pairs.json`.

# Нагрузочная проверка

ТЗ 3.5: «стабильная работа при одновременном использовании десятками пользователей; основные операции — формирование подборки, открытие профиля, поиск — выполняются за приемлемое время».

## Как проверяли

Скрипт `backend/tests/load_api.py` регистрирует нужное число пользователей через API (коды подтверждения берёт из писем, как сквозная проверка), затем запускает их одновременно. Каждый пользователь повторяет обычные действия с паузой 0,5–1,5 секунды между запросами, как человек, который читает страницу:

- работодатели ищут по банку кандидатов (шесть разных запросов), открывают подборку по потребности, свои вакансии и профиль компании;
- кандидаты открывают профиль, вакансии под профиль, задачу недели и уведомления.

Команда на запущенном стенде:

```
make load
# или
docker compose exec -T -e SMOKE_BASE=http://localhost:8000/api/v1 -e MAILPIT_URL=http://mailpit:8025 \
  backend python tests/load_api.py --users 40 --seconds 60
```

## Результат

Стенд с демо-данными (153 кандидата, 9 вакансий), API в двух процессах uvicorn, PostgreSQL 16 и Redis 7 на той же машине: 2 ядра Intel Xeon 2,8 ГГц. Прогон 06.10.2026.

40 пользователей одновременно (20 работодателей, 20 кандидатов), 60 секунд, 2416 запросов, 39,3 запроса в секунду.

| Операция | Запросов | Ошибок 5xx и сетевых | Ответов 429 | p50, мс | p95, мс | p99, мс |
|---|---|---|---|---|---|---|
| Поиск по банку кандидатов | 417 | 0 | 0 | 36 | 69 | 364 |
| Подборка по потребности | 11 | 0 | 0 | 52 | 274 | 344 |
| Профиль кандидата | 293 | 0 | 0 | 10 | 21 | 264 |
| Вакансии под профиль | 314 | 0 | 0 | 11 | 25 | 147 |
| Задача недели | 283 | 0 | 0 | 12 | 26 | 283 |
| Уведомления | 326 | 0 | 0 | 7 | 22 | 313 |
| Вакансии компании | 388 | 0 | 0 | 6 | 17 | 157 |
| Профиль компании | 384 | 0 | 0 | 6 | 15 | 158 |

Ни одной ошибки. 95 % ответов на поиск быстрее 70 мс, на открытие профиля — быстрее 25 мс. Самая тяжёлая операция — подборка по потребности: она ранжирует весь банк по оценке соответствия и считает перцентиль внутри категории, её 95-й перцентиль — 274 мс.

## Что ограничивает масштаб и как его поднять

- **Поиск** ограничен 30 запросами в минуту на работодателя: это защита от выгрузки базы, а не предел производительности. В прогоне лимит не сработал ни разу.
- **Подборка** считается на сервере приложения по кандидатам, прошедшим SQL-фильтры. Для базы ФСП (около 100 тысяч участников) следующий шаг — кешировать подборку по версии потребности: версия уже хранится, и при её изменении кеш сбрасывается.
- **API без состояния**: процессы uvicorn масштабируются горизонтально (`--workers` или реплики контейнера `backend`), состояние сессий входа через ФСП ID и лимиты лежат в Redis.

# Интеграция с реестром ФСП и ФСП ID

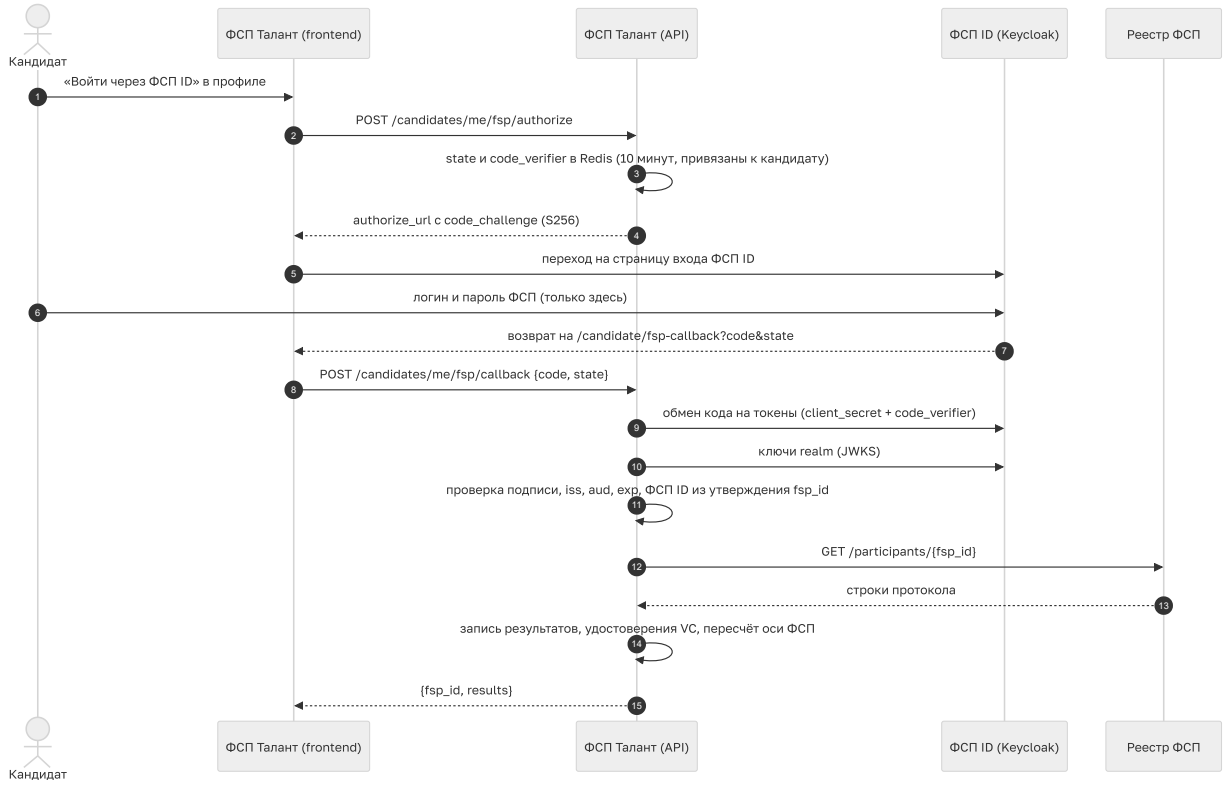
ТЗ не даёт ни API, ни структуры данных ФСП и просит команду решить, что получать и как показывать (раздел 2, 3.4). Ниже — что мы предлагаем Федерации и что уже работает в коде.

Короткий итог:

- **Вход через ФСП ID работает как OpenID Connect.** Платформа — обычный клиент Keycloak, поток кода авторизации с PKCE. В демо роль Keycloak играет помеченная страница этого сайта. Для подключения настоящего ФСП ID код менять не нужно, достаточно настроек в `.env`.
- **Из реестра берём только строки протокола:** соревнование, дисциплина, статус по ЕКП, место, число участников, командность, дата. Пол, контакты и документы не запрашиваем.
- **Результаты ФСП — отдельная ось профиля.** На грейд они не влияют. Кандидат без истории ФСП ничего не теряет: ось просто не учитывается (ТЗ 2.1).

## 1. Вход через ФСП ID (OpenID Connect, Keycloak)

ТЗ: «В дальнейшем ФСП ID будет работать на базе Keycloak, поэтому совместимость с этим решением упростит последующую интеграцию».




Свойства, которые проверяют тесты `backend/tests/unit/test_fsp_identity.py`:

- пароль ФСП вводится только на странице провайдера, платформа его не видит;
- `state` одноразовый и принадлежит одному кандидату: чужой `state` отклоняется;
- код одноразовый и привязан к `code_challenge`: перехваченный код без `code_verifier` этой сессии бесполезен;
- код выдаётся только для зарегистрированного `client_id` и адреса возврата;
- ID-токен проверяется по ключам realm: подпись, издатель, аудитория, срок. Токен для другого клиента, просроченный или подписанный чужим ключом отклоняется;
- вычисление `code_challenge` сверено с примером из RFC 7636.

### Режимы

| FSP_ID_MODE | Где вводят пароль | Что нужно |
|---|---|---|
| `mock` (демо) | Страница `/fsp-id/authorize` этого сайта с пометкой «Демо-провайдер». Реестр FSP2000–FSP2099, пароль `test` | Ничего |
| `oidc` | Страница входа Keycloak Федерации | `FSP_ID_ISSUER`, `FSP_ID_CLIENT_ID`, `FSP_ID_CLIENT_SECRET`, `FSP_ID_CLAIM` |

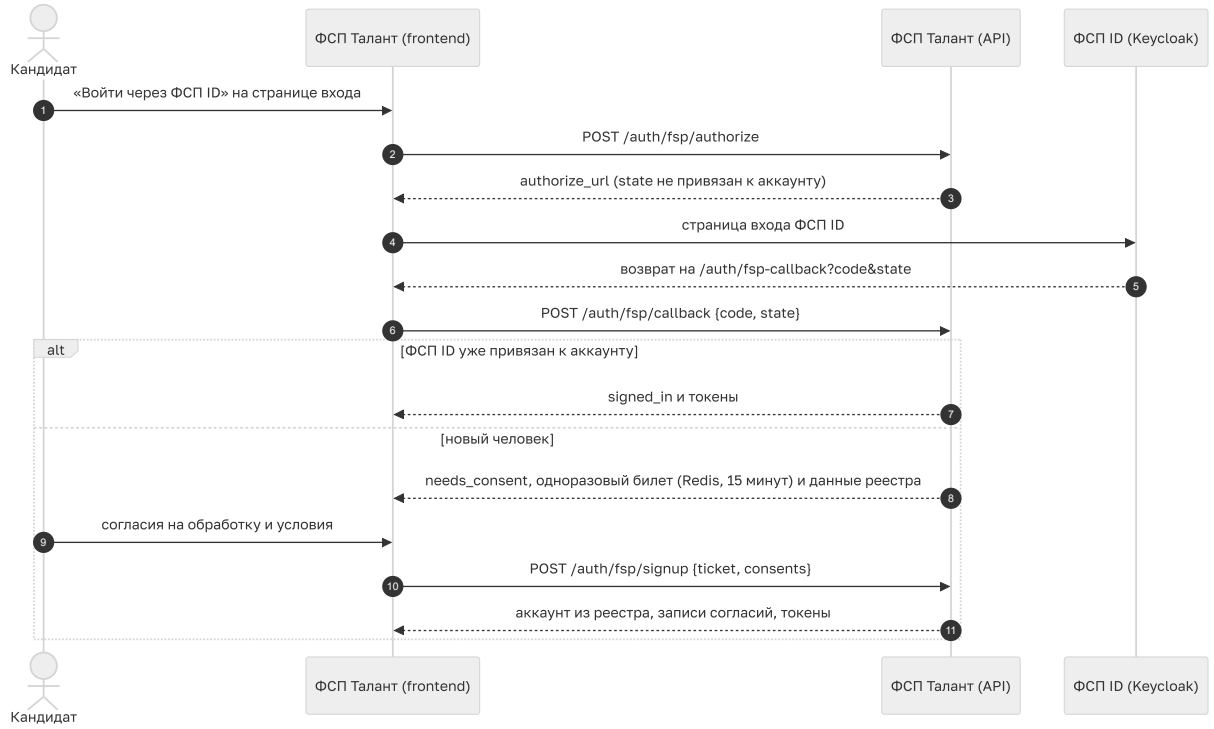
## Что настроить в Keycloak Федерации

1. Клиент `fsp-talent`: Client authentication — On (конфиденциальный клиент), Standard flow — On, остальные потоки выключены.
2. Valid redirect URIs: `https://<адрес стенда>/candidate/fsp-callback` (привязка) и `https://<адрес стенда>/auth/fsp-callback` (вход).
3. Advanced → Proof Key for Code Exchange Code Challenge Method: `S256`.
4. Client scopes → mapper типа «User Attribute»: атрибут пользователя с номером ФСП ID → утверждение `fsp_id` в ID-токене.
5. В `.env` платформы: `FSP_ID_MODE=oidc`, `FSP_ID_ISSUER=https://<keycloak>/realms/<realm>`, `FSP_ID_CLIENT_ID=fsp-talent`, `FSP_ID_CLIENT_SECRET=<секрет клиента>`.

Код: `backend/app/services/fsp/identity.py` (`KeycloakFspIdProvider`, `MockFspIdProvider`), методы API — `backend/app/api/v1/candidates.py` (`/me/fsp/authorize`, `/me/fsp/callback`), демо-провайдер — `backend/app/api/v1/fsp_id_demo.py`.

## Вход в платформу через ФСП ID

Федерация ответила, что ФСП ID — единая точка входа, а вход на хакатоне можно имитировать. Поэтому ФСП ID работает двумя способами: привязка к готовому аккаунту (схема выше) и вход без регистрации по email. Регистрация по email (ТЗ 2.2, п. 4) остаётся.




- Тот же поток кода с PKCE, что и привязка, но свой адрес возврата: `/auth/fsp-callback` (привязка — `/candidate/fsp-callback`). Код, выданный для одного адреса, другой поток не примет. `state` и код одноразовые.

- Без обязательных согласий (`terms`, `pd_candidate`) аккаунт не создаётся: 422 `consent_required`, билет не тратится. Повтор билета — 400 `fsp_ticket_invalid`.
- Такой аккаунт работает без email и пароля: личность подтверждает ФСП ID. Для связи кандидат добавляет Telegram или телефон в профиле, уведомления приходят в колокольчик.
- В Keycloak добавляется второй адрес возврата: `https://<адрес стенда>/auth/fsp-callback`.
- Как показать: «Вход» → «Войти через ФСП ID» → `FSP001` / `test` (аккаунт `fsp001@demo.talent` уже привязан, вход сразу) или любой из FSP2000–FSP2099 / `test` (новый человек, экран согласий).

Код: `backend/app/api/v1/auth.py` (`/fsp/authorize`, `/fsp/callback`, `/fsp/signup`), тесты — `backend/tests/unit/test_fsp_signin.py` (вход привязанного аккаунта, согласия до создания, одноразовые `state` и билет, разделение адресов возврата).

## 2. Контракт реестра ФСП

Предлагаемый контракт — один метод только на чтение. В режиме `FSP_MODE=real` его вызывает `HttpFSPAdapter` (`backend/app/services/fsp/registry.py`). В демо (`FSP_MODE=mock`) те же структуры отдаёт `MockFSPAdapter` с синтетическими протоколами.

```
GET {FSP_API_URL}/participants/{fsp_id}
Authorization: Bearer {FSP_API_KEY}
Accept: application/json
```

```
{
  "fsp_id": "FSP001",
  "display_name": "Иван Иванов",
  "region": "Томская область",
  "birth_date": "2003-05-12",
  "results": [
    {
      "event_id": "CR2025-ALG",
      "event_name": "Чемпионат России по спортивному программированию 2025",
      "discipline": "algorithmic",
      "competition_status": "russia_championship",
      "place": 3,
      "total_participants": 180,
      "regions_count": 52,
      "is_team": false,
      "team_size": 1,
      "role": "participant",
      "event_date": "2025-11-20",
      "protocol_url": "https://…"
    }
  ]
}
```

| Ответ | Что делает платформа |
|---|---|
| 200 | Записывает строки протокола, выдаёт по каждой подписанное удостоверение, пересчитывает ось ФСП |
| 404 | «Протоколы ФСП не найдены» (`fsp_not_found`), профиль остаётся без оси ФСП |
| другой код или неверная структура | 503 «Реестр ФСП сейчас не отвечает», ранее загруженные данные не трогаются |

Строка с неизвестной дисциплиной или местом вне `1…total_participants` пропускается, а не угадывается.

### Поля и зачем они нужны

| Поле | Зачем | Где используется |
|---|---|---|
| `discipline` | Какая из шести дисциплин ФСП: алгоритмическое, продуктовое, системы ИБ, БАС, робототехника, системы ИИ | Ось ФСП по дисциплинам, фильтр поиска |
| `competition_status` | Статус соревнования по ЕКП | Разряд-эквивалент по нормам ЕВСК для справки и фильтра; в оценку не входит |
| `place`, `total_participants` | Класс результата: победитель, призёр, топ-10 %, верхняя половина, участие | Качество результата |
| `regions_count` | Ценз ЕВСК для МС и КМС | Разряд-эквивалент |
| `is_team`, `team_size` | Командный результат | Вес доказательства делится на √(размер команды): уверенность ниже, уровень не снижается |
| `event_date` | Давность | Затухание с полупериодом 18 месяцев |
| `birth_date` | Совершеннолетие | Профили младше 18 лет скрыты от работодателей без согласия законного представителя |
| `region` | Подпись в профиле | В скор подбора не входит |
| `display_name` | Подпись, если кандидат не указал имя сам | Видна работодателю только после согласия кандидата |

**Не запрашиваем:** пол, контакты, паспортные данные, место учёбы. Принцип минимизации из ст. 5 152-ФЗ: берём только то, что нужно для оценки.

Значения `competition_status`: `world`, `russia_championship`, `russia_cup`, `russia_youth`, `all_russian`, `federal_district`, `regional`, `other`.

### Аутентификация сервиса

В демо-контракте это ключ `Bearer`. В продакшене удобнее токен Keycloak по `client_credentials` того же клиента `fsp-talent` с отдельной ролью «чтение протоколов». Тогда доступ к реестру отзывается в одном месте вместе с входом.

## 3. Обновление данных

Протоколы по-прежнему загружаются при привязке ФСП ID. Повторно их забирает почасовая задача `sync_fsp_changes`: она спрашивает реестр, кто изменился после сохранённой отметки, и дописывает только новые строки уже привязанным участникам.

```
GET {FSP_API_URL}/participants/changes?since=2026-10-01T00:00:00Z
→ {"fsp_ids": ["FSP001", "FSP017"], "next_since": "2026-10-06T12:00:00Z"}
```

Отметку хранит Redis (`fsp:changes:since`). Первый запуск смотрит на два часа назад. Повтор той же строки безопасен: пара (участник, соревнование) в таблице `results` уникальна, уже лежащая строка не пишется второй раз. В демо ту же выборку отдаёт `MockFSPAdapter`. Если Федерации удобнее push, подойдёт вебхук «протокол утверждён» с той же структурой и подписью HMAC.

## 4. Как результат становится доказательством

1. Строка протокола → класс результата по месту и размеру поля (`result_class`, методика `v3.1.0`): победитель 1,00 · призёр (2–3 место) 0,85 · топ-10 % 0,65 · верхняя половина 0,45 · участие 0. Победитель и призёр — при шести участниках и больше. В сумму дисциплины входят пять сильнейших событий. Статус соревнования не учитывается: по совету Федерации все соревнования одинаково важны.

2. Вес результата = затухание с полупериодом 18 месяцев / √(размер команды). Вклад внутри команды не разделяется: команда снижает уверенность, а не уровень.
3. По дисциплине — байесовское сжатие к среднему и нижняя граница уровня (LCB). Один случайный успех не обгоняет стабильную серию. Участие без призов ниже априорного среднего, поэтому профиль нельзя набрать онлайн-участиями: двадцать участий дают LCB около 0,21, две победы — около 0,55.
4. Ось ФСП влияет на место кандидата внутри категории и никогда — на грейд: грейд даёт только тест специализации. Разряд-эквивалент по ЕВСК (`rank_equivalent`) показывается в профиле, удостоверении и фильтре «Разряд не ниже», но в оценку не входит. Формулы — в README и `backend/app/services/scoring/engine.py`.
5. По каждой строке выдаётся удостоверение W3C Verifiable Credentials 2.0 (`eddsa-jcs-2022`). Любой может проверить подпись: `GET /api/v1/public/issuer`, `POST /api/v1/public/credentials/verify`.

Сейчас издатель удостоверений — платформа: она подтверждает, что результат получен из системы ФСП. В продакшене издателем может стать сама Федерация со своим ключом, формат при этом не меняется.

## 5. Что видит работодатель

Карточка в выдаче анонимна: статус соревнования, год и диапазон процентиля («топ-5 %»). Названия события и точного места в ней нет, чтобы по ним нельзя было найти человека в открытом протоколе. После того как кандидат принял приглашение или откликнулся сам, работодатель видит имя и контакты. Каждый просмотр записывается в журнал доступа.

# Защита от фиктивных вакансий и интеграция с ATS

ТЗ (раздел 2): «Будет плюсом: механизм защиты от фиктивных вакансий и недобросовестных работодателей, интеграция с внутренними ATS-системами работодателей». От команды здесь ожидают концепцию и архитектурный подход. Ниже отмечено, что из этого уже работает в коде, а что предлагается следующим шагом.

## 1. Защита от фиктивных вакансий и недобросовестных работодателей

### От чего защищаемся

| Угроза | Чем вредит |
|---|---|
| Фиктивная вакансия ради сбора контактов | Утечка персональных данных, спам кандидатам |
| Вилка-приманка: в приглашении 300 тыс., на интервью 120 тыс. | Подрывает главное отличие платформы: условия видны до общения |
| Массовая рассылка одного шаблона всем подряд | Обратная механика превращается в спам |
| Компания-однодневка или выдача себя за другую компанию | Кандидат доверяет бренду, которого нет |
| Сбор открытых протоколов ФСП, чтобы найти человека | Обход анонимности карточки |

### Что уже работает

| Мера | Где в коде |
|---|---|
| Контакты кандидата закрыты до его согласия: принятого приглашения или собственного отклика. Каждый просмотр пишется в журнал доступа | `app/api/v1/offers.py`, `app/api/v1/vacancies.py`, таблица `access_logs` |
| Кандидат закрывает контакты для одной компании, не отзывая согласие целиком: «Приглашения» → «Закрыть контакты для этой компании». Закрываются все принятые приглашения и отклики этой компании, пока она не получит новое принятое приглашение; компания видит 403 `contacts_revoked`, попытки после закрытия пишутся в журнал | `POST /candidates/me/offers/{id}/revoke-contacts`, `consent_logs`, `access_logs` |
| Переписка открывается только после принятия приглашения, после закрытия контактов — только чтение. Не больше 60 сообщений в час, уведомление без текста сообщения | `app/api/v1/messages.py` |
| Зарплатная вилка «от–до» обязательна в приглашении и вакансии, «до» не меньше «от», есть верхняя граница | `OfferCreate`, `VacancyIn` |
| Не больше 50 приглашений в сутки от одной компании | `_OFFERS_PER_DAY`, `app/api/v1/offers.py` |
| Одно активное приглашение на пару «компания — кандидат» (уникальный индекс) | миграции, `app/services/offers.py` |
| Приглашение истекает через 14 дней, кандидат видит срок | `expires_at`, задача `expire_offers` |
| ИНН проверяется по контрольной сумме ФНС без внешних запросов. Кандидат видит в приглашении, указан ли ИНН | `app/services/inn.py`, `company_profile.inn_given` |
| Профиль компании (сфера, сайт, описание) кандидат видит до принятия решения | `/candidate/offers`, «О компании» |
| Карточка в выдаче анонимна: статус соревнования, год и диапазон процентиля без названия события и места | `app/services/search/pipeline.py` |
| Уведомления не несут данных другой стороны | `app/services/notify.py` |
| Лимиты на вход, регистрацию и письма. Если хранилище лимитов недоступно, эти методы закрываются, а не пропускают запросы | `app/core/ratelimit.py` |

### Что предлагаем дальше

**Уровни доверия к компании.** Кандидат видит уровень рядом с названием, поиск и лимиты зависят от него.

| Уровень | Как получить | Что даёт |
|---|---|---|
| 0. Новая | Почта подтверждена | 10 приглашений в сутки, у кандидата пометка «новая компания» |
| 1. Реквизиты | ИНН прошёл контрольную сумму, по ЕГРЮЛ (ФНС, API «ЕГРЮЛ/ЕГРИП») компания действующая, название совпадает | 50 приглашений в сутки |
| 2. Домен | Почта на домене сайта компании, сайт отвечает и указан в ЕГРЮЛ или подтверждён записью DNS TXT | Метка «реквизиты и домен подтверждены» |
| 3. Партнёр ФСП | Компания — партнёр или спонсор соревнований ФСП | Метка Федерации, приоритет в «задаче недели» |

**Сигналы от кандидатов.** После интервью кандидат отвечает на один вопрос: «Условия совпали с приглашением?» и может нажать «Пожаловаться» с причиной (вилка не совпала, вакансии нет, спам). Доля «не совпали» и жалоб считается по компании со сглаживанием к среднему, как ось ФСП. Так новая компания с одной жалобой не тонет, а системная проблема видна.

**Автоматические правила.** Порог жалоб замораживает рассылку до ручной проверки. Вилка ниже рыночной медианы категории в 2 раза или выше в 3 раза отправляется на проверку: медиану платформа считает сама по приглашениям той же «специализации × грейда». Одинаковый текст в сотне приглашений за час снижает лимит.

**Решение остаётся за человеком.** Автоматика только ставит в очередь модерации и ограничивает лимит. Блокировка — решение модератора, компания видит причину и может её оспорить.

## 2. Интеграция с ATS работодателя

Цель — чтобы рекрутер работал в своей ATS (Хантфлоу, Talantix, E-Staff, Greenhouse), а платформа была источником кандидатов с подтверждённой категорией.

### Принципы

- **Платформа отдаёт события, а не базу.** ATS получает только кандидатов, которые сами согласились на контакт: приняли приглашение или откликнулись. До этого из платформы уходит только анонимная категория.
- **Данные передаются по тем же правилам, что видит работодатель в кабинете.** Отзыв согласия и закрытие контактов для компании отправляют событие удаления, и ATS этой компании обязана удалить контакты (ст. 21 152-ФЗ).
- **Два направления:** платформа → ATS (вебхуки и экспорт) и ATS → платформа (статусы найма для воронки и для будущей калибровки подбора).

### Вебхуки (платформа → ATS)

Работодатель указывает в кабинете адрес и получает секрет. Каждое событие — `POST` с телом JSON и заголовком `X-FSP-Talent-Signature: sha256=<HMAC(secret, body)>`. Доставка с повтором (1 мин, 5 мин, 30 мин, 2 ч), идемпотентность по `event_id`.

| Событие | Когда | Что внутри |
|---|---|---|
| `offer.accepted` | Кандидат принял приглашение | Приглашение, категория, подтверждения, контакты, PDF-профиль по ссылке |
| `offer.declined` | Кандидат отклонил | Приглашение, причина, если указана |
| `application.created` | Кандидат откликнулся на вакансию | Вакансия, категория, контакты |
| `consent.withdrawn` | Кандидат отозвал согласие | `candidate_ref` для удаления |

| Событие | Когда | Что внутри |
|---|---|---|
| `contacts.revoked` | Кандидат закрыл контакты для этой компании | `candidate_ref` для удаления у этой компании |

```
{
  "event_id": "8c1d…",
  "type": "offer.accepted",
  "created_at": "2026-10-06T12:00:00Z",
  "employer_id": "…",
  "candidate": {
    "ref": "c_5f2…",
    "name": "…", "email": "…", "telegram": "…",
    "category": {"specialization": "backend", "grade": "middle", "grade_source": "assessed"},
    "skills": [{"name": "Python", "source": "test", "confirmed_at": "2026-09-30"}],
    "fsp": [{"discipline": "algorithmic", "lcb": 0.42, "best_rank": "KMC"}],
    "profile_pdf": "https://…/api/v1/…/profile.pdf"
  },
  "offer": {"role": "Backend-разработчик", "salary_from": 230000, "salary_to": 300000, "work_format":
"remote"}
}
```

Формат кандидата совместим по смыслу с JSON Resume. Для ATS, которые принимают только резюме-файл, есть стандартизированный PDF-профиль: он уже формируется (`/api/v1/candidates/me/profile.pdf`).

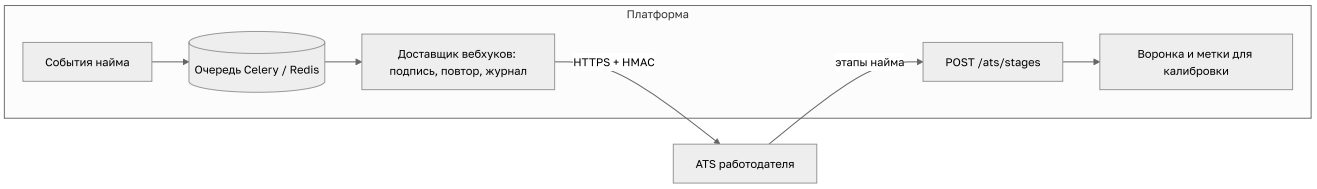
### Статусы найма (ATS → платформа)

`POST /api/v1/ats/stages` с `candidate_ref` и этапом: `screening`, `interview`, `offer`, `hired`, `rejected`. Платформа показывает этапы в воронке работодателя и копит размеченные пары «потребность — кандидат — итог». Это и есть данные для калибровки подбора: сейчас валидация идёт на синтетике, потому что меток найма у хакатона нет (`docs/VALIDATION_MATCHING.md`). Скрипт `python -m app.validation.matching --import pairs.json` уже принимает такие пары.

### Пример: Хантфлоу

1. Вебхук `offer.accepted` → создание кандидата методом API Хантфлоу «добавить кандидата» с источником «ФСП Талант» и прикреплённым PDF-профилем.
2. Привязка к вакансии Хантфлоу по `need_id` или `vacancy_id`, который работодатель сопоставил в кабинете.
3. Вебхук Хантфлоу о смене этапа → `POST /api/v1/ats/stages` платформы.

### Архитектура




Доставщик — отдельная задача Celery: очередь и повторы уже есть в стенде. Журнал доставок виден работодателю в кабинете.

# Коды ошибок API

Любая ошибка возвращается в одном формате:

```
{"error": {"code": "email_not_verified", "message": "Email не подтверждён: введите код из письма",
"details": {"email": "a@b.ru", "role": "participant"}},
 "detail": "Email не подтверждён: введите код из письма"}
```

`detail` оставлен для клиентов, написанных под формат FastAPI по умолчанию. Для ошибок валидации (422) `detail` и `error.details` содержат список полей. У 429 есть заголовок `Retry-After`.

Список ниже сверяется с кодом тестом `tests/unit/test_identity.py`.

| Код | Значение |
|---|---|
| `bad_request` | HTTP 400 |
| `unauthorized` | HTTP 401 |
| `forbidden` | HTTP 403 |
| `not_found` | HTTP 404 |
| `method_not_allowed` | HTTP 405 |
| `conflict` | HTTP 409 |
| `gone` | HTTP 410 |
| `validation_error` | HTTP 422 |
| `rate_limited` | HTTP 429 |
| `internal_error` | HTTP 500 |
| `unavailable` | HTTP 503 |
| `invalid_credentials` | Неверный email или пароль |
| `email_not_verified` | Email не подтверждён: введите код из письма |
| `email_taken` | Аккаунт с таким email уже есть |
| `verification_invalid` | Код или ссылка не подходят |
| `verification_expired` | Срок кода истёк, запросите новый |
| `verification_attempts` | Слишком много неверных попыток, запросите новый код |
| `resend_cooldown` | Новый код можно запросить через минуту |
| `consent_required` | Нужно согласие на обработку данных |
| `consent_unknown` | Нет такого документа |
| `consent_withdrawn` | Кандидат отозвал согласие на обработку данных: контакты недоступны |
| `delete_confirm` | Чтобы удалить аккаунт, введите слово «удалить» |
| `fsp_invalid` | Неверные учётные данные ФСП |
| `fsp_taken` | Этот ФСП ID уже привязан к другому аккаунту |
| `fsp_already_linked` | К аккаунту уже привязан ФСП ID |
| `fsp_not_found` | Протоколы ФСП не найдены |
| `survey_required` | Сначала пройдите опрос: выберите специализацию |
| `grade_locked` | Подтверждённый грейд пока нельзя менять |
| `retake_cooldown` | Пересдача доступна позже |

| Код | Значение |
|---|---|
| `question_bank_empty` | Не хватает заданий для теста |
| `session_closed` | Сессия теста уже завершена |
| `session_expired` | Время сессии истекло |
| `item_token_invalid` | Задание не принадлежит этой сессии |
| `offer_closed` | Приглашение уже обработано |
| `offer_expired` | Срок приглашения истёк |
| `contacts_hidden` | Контакты откроются после согласия кандидата |
| `salary_required` | Укажите зарплатную вилку от и до |
| `need_not_found` | Потребность не найдена |
| `contacts_revoked` | Кандидат закрыл контакты для вашей компании |
| `task_closed` | Задание закрыто или срок сдачи прошёл |
| `task_answered` | Ответ на это задание уже отправлен |
| `fsp_ticket_invalid` | Вход через ФСП ID устарел: начните вход заново |

# Дизайн-система «Протокол»

Интерфейс ФСП Талант построен на брендбуке Федерации спортивного программирования. Главный объект продукта — подборка кандидатов, поэтому язык интерфейса взят из протокола соревнований: место, категория, подтверждения, табличные цифры, тонкие линейки.

## Что взято из брендбука ФСП

Источник: fsp-russia.ru/press/brandbook.

| Элемент брендбука | Как используется |
|---|---|
| `#402FFF` — основной | Действия, ссылки, фокус, подтверждённый грейд, линии сети на карте, знак продукта |
| `#EC1D35` — красный | Полоса бренда над шапкой, точка в знаке, узел «Москва» на карте. Кнопка «Пригласить» на шаг темнее (`#D9172F`): так белый текст проходит AA |
| `#EDEDED` — светло-серый | Фон страниц светлой темы, основной текст тёмной |
| `#C8C9CA` — серый | Сильные линейки, точки карты, рамки полей |
| `#1B1C21` — тёмный | Основной текст светлой темы, панели тёмной, финальная полоса на главной |
| Шрифт JetBrains Mono (OFL 1.1) | Заголовки, цифры, места, грейды, подписи карты. Длинный русский текст набран Golos Text, чтобы читаться без усталости |
| Карта России и соединительные линии | Главный графический элемент главной: точечная карта и сеть региональных центров, которая один раз прорисовывается при загрузке |

Цветовые токены лежат в `frontend/src/index.css` в виде RGB-троек, поэтому Tailwind умеет накладывать прозрачность (`bg-accent/45`). Имена в `tailwind.config.ts`: `paper`, `panel`, `sunken`, `ink`, `ink-2`, `ink-3`, `line`, `line-strong`, `accent`, `invite`, `ok`, `warn`, `medal`, `brand-red`.

## Правила

- **Одно главное действие на экран.** Это синяя кнопка. Красная кнопка только одна — «Пригласить», и больше ничего красного в интерфейсе нет. Золото только у разрядов МС и КМС.
- **Категория — это плашка «специализация × грейд».** Если грейд подтверждён тестом, его ячейка залита синим. Если заявлен — серая.
- **Оценка всегда с разбором.** Под карточкой показаны полосы только тех факторов, которые участвуют в оценке соответствия. Остальные перечислены строкой «Не учитываются». Если критериев нет, подпись меняется на «сила профиля»: так точнее, чем «совпадение».
- **Движение одно и осмысленное.** На главной один раз прорисовывается сеть регионов. Остальная анимация отвечает на действия: раскрытие разбора, заполнение полос. При `prefers-reduced-motion` всё статично.
- **Ничего внешнего.** Шрифты, Swagger UI и карта лежат в проекте. CSP стенда: `script-src 'self'`, `font-src 'self'`.

## Компоненты

- `hero-ink` — фирменная поверхность: `#402FFF` и точечная фактура карты, затухающая к левому краю. Одна на экран, только для главного итога: первый экран главной, категория кандидата, воронка работодателя, первое место рейтинга, итог теста. Внутри — `btn-on-hero` (белая), `btn-on-hero-ghost`, `chip-on-hero`, `on-hero-muted`.
- `btn-invite-quiet` — «Пригласить» в длинных списках: красный контур, заливка при наведении. Сплошная красная `btn-invite` остаётся только в окне отправки приглашения.
- **Знак компании** — первая буква названия моноширинным шрифтом на `accent-soft`. Встречается в приглашениях, вакансиях и превью компании.
- `plural(n, [one, few, many])` — склонение числительных во всех счётчиках.
- `Meter` — полоса со значением; без видимой подписи у неё обязателен `ariaLabel`.

## Контраст (WCAG 2.2 AA)

Посчитан по формуле относительной яркости:

| Пара | Контраст |
|---|---|
| `#402FFF` на белом | 6,84 |
| `#402FFF` на `#EDEDED` | 5,85 |
| Белый на `#D9172F` (кнопка «Пригласить») | 5,10 |
| `#1B1C21` на `#EDEDED` | 14,53 |
| Вторичный текст `#63656E` на `#EDEDED` | 4,96 |
| Синий тёмной темы `#8A7FFF` на `#121317` | 5,80 |
| Красный тёмной темы `#FF5A6B` и тёмный текст на нём | 6,12 |
| Золото разрядов `#8A5A00` на `#FCF3DC` | 5,36 |

Проверка axe-core в двух темах: 0 нарушений уровней serious и critical (UI_AUDIT_2026-10-06.md).

## Карта

`frontend/src/assets/russia-network.json` (25 КБ) содержит 1 772 точки и 30 региональных центров, соединённых минимальным остовным деревом от Москвы. Как она собрана:

- геометрия — Natural Earth 1:50m (public domain) из пакета `world-atlas`;
- проекция — коническая равновеликая, параллели 52° и 68°;
- генератор — `frontend/scripts/gen-russia-network.mjs`;
- зависимости генератора ставятся отдельно и в `package.json` не попадают.

На странице карта — один SVG из трёх путей: точки нарисованы как `M x y h0` с круглым концом.

## Практики, на которые опирались

- **Схемы, сетки и моноширинный шрифт вместо декоративного стекла.** Тренд описан в Tubik, UI Design Trends 2026. Пример — Vercel Geist: моно для данных, высокий контраст, сетка.
- **Движение как обратная связь, и управляемое.** Учитывается `prefers-reduced-motion`; гибкий кегль заголовка задан через `clamp()`.
- **Анонимная карточка до согласия и зарплата в приглашении.** Так устроены платформы обратного найма. Мы дополняем их разбором скора и подтверждением грейда тестом.